

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN LẬP TRÌNH TÍNH TOÁN

CHƯƠNG TRÌNH QUẢN LÝ GIA PHẢ

Giảng viên hướng dẫn:

TS. Nguyễn Văn Hiệu

Sinh viên thực hiện:

Nguyễn Đình Hải Đăng

Lớp: 25T_KHDL

Lê Bá Sơn

Lớp: 25T_KHDL

Đà Nẵng, tháng 6 năm 2026

MỤC LỤC

Danh mục hình ảnh	iv
Mở đầu	v
1 TỔNG QUAN ĐỀ TÀI	1
1.1 Mục đích thực hiện đề tài	1
1.2 Mục tiêu đề tài	1
1.3 Phạm vi và đối tượng nghiên cứu	1
1.4 Phương pháp nghiên cứu	1
1.5 Cấu trúc đồ án	2
2 CƠ SỞ LÝ THUYẾT	2
2.1 Ý tưởng	2
2.2 Cơ sở lý thuyết	3
2.2.1 Cấu trúc dữ liệu cây	3
2.2.2 Biểu diễn cây bằng mô hình con đầu - anh em kế tiếp	4
2.2.3 Duyệt cây và quản lý bộ nhớ	5
3 TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN	6
3.1 Phát biểu bài toán	6
3.2 Cấu trúc dữ liệu	7
3.2.1 Thiết kế chung	7
3.2.2 Cài đặt các ràng buộc cần thiết	8
3.3 Thuật toán và đánh giá thuật toán	8
3.3.1 Thuật toán Duyệt và Tìm kiếm	8
3.3.2 Thuật toán Xác định quan hệ	9
3.3.3 Thuật toán truy vấn theo danh xưng	12
3.3.4 Thuật toán lưu trữ và phục hồi	14
3.3.5 Thuật toán thống kê dòng họ	16
4 CHƯƠNG TRÌNH VÀ KẾT QUẢ	17
4.1 Tổ chức chương trình	17
4.1.1 Kiến trúc tổng thể của chương trình	17
4.1.2 Cấu trúc tệp tin và phân rã chức năng	17

4.2	Ngôn ngữ cài đặt	18
4.2.1	Lý do sử dụng ngôn ngữ C++	18
4.2.2	Môi trường phát triển và công cụ hỗ trợ	18
4.3	Kết quả	19
4.3.1	Giao diện chính của chương trình	19
4.3.2	Kết quả thực thi chương trình	19
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	21
5.1	Kết luận	21
5.2	Hướng phát triển	21
	Tài liệu tham khảo	23
	Phụ lục	24

DANH MỤC HÌNH ẢNH

1	Hình minh hoạ mô hình biểu diễn cây theo cấu trúc con đầu - anh em kế tiếp	4
2	Mã giả mô tả thuật toán thêm một người con vào cây gia đình	5
3	Đoạn mã C++ thể hiện logic duyệt cây theo chiều sâu và giải phóng bộ nhớ.	6
4	Sơ đồ ERD mô tả thực thể Person và các mối quan hệ tự thân trong cấu trúc cây LCRS	7
5	Mô phỏng quy trình thực hiện truy vấn theo Định danh bằng cấu trúc Bảng băm	9
6	Mã giả quy trình thực hiện truy vấn theo Định danh bằng cấu trúc Bảng băm	9
7	Mã giả và hình vẽ minh hoạ cho pha 1 của thuật toán.	10
8	Mã giả và hình vẽ minh hoạ cho pha 2 của thuật toán	11
9	Mã giả mô tả pha thứ nhất của thuật toán truy vấn theo danh xưng . . .	13
10	Mã giả mô tả pha thứ hai của thuật toán truy vấn theo danh xưng	13
11	Mã giả mô tả thuật toán phẳng hoá cây để lưu file	15
12	Sơ đồ mô tả cấu trúc chương trình và các tệp tin	18
13	Giao diện chính của chương trình	19
14	Tổng hợp các màn hình thực thi các chức năng chính của hệ thống quản lý gia phả	19
14	Tổng hợp các màn hình thực thi các chức năng chính của hệ thống quản lý gia phả (<i>tiếp theo</i>)	20

MỞ ĐẦU

Trong bối cảnh công nghệ thông tin ngày càng phát triển mạnh mẽ, việc ứng dụng công nghệ vào giải quyết các bài toán thực tiễn đóng vai trò quan trọng. Trong đó, nhiều văn hoá truyền thống của dân tộc Việt Nam được “số hoá”, tạo cơ hội cho những người trẻ - những người sống trong thời đại công nghệ được tiếp cận với truyền thống văn hoá nước nhà theo một cách mới mẻ và thú vị hơn.

Xuất phát từ ý tưởng số hóa và tổ chức thông tin về các mối quan hệ huyết thống trong gia đình, nhóm chúng em lựa chọn thực hiện đề tài “**Chương trình Quản lý gia phả**”. Mục tiêu của chương trình là xây dựng một hệ thống cho phép lưu trữ, quản lý và truy vấn thông tin các thành viên trong gia đình dưới dạng cấu trúc cây, từ đó hỗ trợ thực hiện các thao tác như thêm thành viên, tìm kiếm quan hệ họ hàng, duyệt cây gia phả và hiển thị thông tin một cách có hệ thống. Thông qua việc xây dựng chương trình, nhóm có cơ hội vận dụng các kiến thức cơ bản về cấu trúc dữ liệu, thuật toán và lập trình để giải quyết bài toán quản lý thông tin có cấu trúc phân cấp - đúng như tinh thần của phương pháp *học tập qua dự án*.

Trong quá trình thực hiện dự án, nhóm đã nhận được sự hướng dẫn của TS. Nguyễn Văn Hiệu, giảng viên phụ trách học phần. Nhóm xin chân thành cảm ơn thầy đã định hướng và góp ý để nhóm có thể hoàn thành dự án này.

Mặc dù rất mong muốn thực hiện một chương trình hoàn chỉnh cả về nền tảng lý thuyết và phương pháp thực hiện, cũng như mong muốn có thêm phản hồi từ người dùng về chất lượng chương trình, tuy nhiên vì thời gian và trình độ còn hạn chế, vì vậy chương trình và báo cáo chắc chắn sẽ còn tồn tại những thiếu sót nhất định, kính mong quý thầy cô đóng góp ý kiến cho chương trình và báo cáo để nhóm rút kinh nghiệm cho những dự án PBL tiếp theo trong quá trình học tập tại trường.

Xin trân trọng cảm ơn.

1. TỔNG QUAN ĐỀ TÀI

1.1. Mục đích thực hiện đề tài

Trong văn hóa Á Đông, việc lưu giữ gia phả là một truyền thống quý báu để duy trì mối liên hệ huyết thống giữa các thế hệ. Tuy nhiên, các phương pháp lưu trữ truyền thống trên giấy tờ đang bộc lộ nhiều hạn chế về khả năng tìm kiếm, cập nhật và tính toán các mối quan hệ phức tạp. Mục đích của đề tài là xây dựng một hệ thống phần mềm chuyên dụng nhằm số hóa gia phả, ứng dụng các cấu trúc dữ liệu tối ưu để quản lý thông tin dòng họ một cách khoa học và bền vững.

1.2. Mục tiêu đề tài

Đề tài tập trung vào việc hiện thực hóa các mục tiêu kỹ thuật và ứng dụng sau:

- Xây dựng cấu trúc lưu trữ: Thiết kế và triển khai cây gia phả dựa trên cấu trúc cây Left-Child Right-Sibling (LCRS) để tối ưu hóa việc quản lý số lượng nút con không xác định.
- Phát triển thuật toán truy vấn: Ứng dụng thuật toán Lowest Common Ancestor (LCA) để tự động xác định chính xác các quan hệ họ hàng dựa trên khoảng cách thế hệ.
- Đảm bảo tính bền vững của dữ liệu: Triển khai cơ chế lưu trữ và phục hồi dữ liệu thông qua tệp nhị phân (.dat/.bin) để tăng tốc độ truy xuất và bảo mật thông tin.
- Tối ưu hóa giao diện người dùng: Cung cấp giao diện console trực quan hỗ trợ điều hướng bằng phím mũi tên và chức năng tìm kiếm, lọc kết quả thông minh.

1.3. Phạm vi và đối tượng nghiên cứu

- Đối tượng nghiên cứu: Các cấu trúc dữ liệu dạng cây, thuật toán duyệt cây, thuật toán tìm tổ tiên chung gần nhất (LCA) và kỹ thuật xử lý tệp nhị phân trong C++.
- Phạm vi nghiên cứu: Hệ thống tập trung quản lý thông tin cá nhân và quan hệ huyết thống theo dòng cha (paternal lineage). Dữ liệu được giới hạn trong phạm vi một dòng họ với các thông tin cơ bản như: họ tên, giới tính, ngày sinh, nghề nghiệp và mối quan hệ trực hệ/bàng hệ.

1.4. Phương pháp nghiên cứu

- Nghiên cứu lý thuyết: Phân tích các tài liệu về cấu trúc dữ liệu và giải thuật, đặc biệt là các biến thể của cây tổng quát và cách chuyển đổi sang cây nhị phân.

- Phương pháp thực nghiệm: Xây dựng phần mềm mẫu (prototype) bằng ngôn ngữ C++, thực hiện các kiểm thử (unit test) trên bộ dữ liệu gia phả thực tế để đánh giá tính chính xác của thuật toán LCA và hiệu năng lưu trữ file.

- Quy trình phát triển: Áp dụng mô hình phát triển phần mềm theo dự án (PBL), từ bước phân tích yêu cầu, thiết kế kiến trúc đa file đến triển khai và tối ưu hóa.

1.5. Cấu trúc đồ án

Nội dung của đồ án chia làm 5 chương:

- Chương 1 - Giới thiệu chung: Trình bày lý do chọn đề tài, mục tiêu và phạm vi nghiên cứu.
- Chương 2 - Cơ sở lý thuyết: Phân tích cấu trúc cây LCRS, thuật toán LCA và các kỹ thuật xử lý tệp trong C++.
- Chương 3 - Tổ chức cấu trúc dữ liệu và thuật toán: Cách thức tổ chức dữ liệu bài toán và hướng triển khai các thuật toán cốt lõi của chương trình.
- Chương 4 - Chương trình và kết quả: Mô tả kiến trúc tổng thể, cấu trúc tệp tin, hiển thị giao diện chương trình, kết quả chạy thực tế.
- Chương 5 - Kết luận và hướng phát triển: Kết luận về đề tài và hướng thực hiện trong tương lai.

2. CƠ SỞ LÝ THUYẾT

2.1. Ý tưởng

Về cấu trúc dữ liệu cốt lõi, nhóm sử dụng Cấu trúc liên kết động (Dynamic Linked Structure). Mỗi thành viên trong gia phả được đại diện bởi một nút (Node) trong bộ nhớ Heap.

Định nghĩa Nút (Person Node): Mỗi nút chứa các thành phần thông tin cá nhân (Data) và các thành phần liên kết (Pointers).

Cơ chế liên kết: Nhóm sử dụng mô hình Left-Child Right-Sibling (LCRS) để biểu diễn cây đa phân, cụ thể:

- pLeftChild: Con trỏ quản lý địa chỉ của người con đầu tiên.
- pRightSibling: Con trỏ quản lý địa chỉ của người em kế tiếp trong cùng một thế hệ.

- **pParent**: Con trỏ ngược về nút cha để tối ưu hóa thuật toán truy vết quan hệ (LCA).

Về quản lý bộ nhớ động, ý tưởng chính là tối ưu hóa tài nguyên hệ thống. Chỉ khi thêm một thành viên mới, chương trình mới sử dụng lệnh `new` để “xin” cấp phát bộ nhớ. Điều này giúp hệ thống có thể quản lý gia phả với quy mô không giới hạn (tùy thuộc vào RAM) thay vì bị giới hạn bởi kích thước mảng khai báo trước. Ngoài ra, nhóm cài đặt cơ chế đệ quy để thu hồi bộ nhớ (`delete`) khi xóa một nhánh hoặc khi kết thúc chương trình, đảm bảo không xảy ra hiện tượng rò rỉ bộ nhớ (Memory Leak).

Về các thuật sử dụng trên cây con trỏ:

1. Duyệt cây: Nhóm sử dụng thuật toán Duyệt theo chiều sâu (DFS) thông qua đệ quy để hiển thị sơ đồ gia phả. Việc di chuyển giữa các nút thực chất là việc truy cập vào địa chỉ bộ nhớ được lưu trữ trong các con trỏ liên kết.

2. Định vị đối tượng: Thay vì tìm kiếm theo chỉ số (Index), nhóm xây dựng hàm tìm kiếm trả về địa chỉ vùng nhớ của đối tượng. Địa chỉ này sau đó được dùng làm tham số đầu vào cho các thao tác cập nhật hoặc thêm con cháu, giúp tốc độ truy cập dữ liệu đạt mức tối ưu.

2.2. Cơ sở lý thuyết

2.2.1. Cấu trúc dữ liệu cây

Cây (Tree) là một cấu trúc dữ liệu phi tuyến tính, được sử dụng để biểu diễn các mối quan hệ phân cấp giữa các đối tượng. Trong cấu trúc cây, mỗi phần tử được gọi là một nút (node), các nút được liên kết với nhau thông qua quan hệ cha - con. Một cây có các đặc điểm cơ bản sau:

- Có một nút gốc (root), là nút không có cha.
- Mỗi nút (trừ nút gốc) có duy nhất một nút cha.
- Mỗi nút có thể có không hoặc nhiều nút con.
- Không tồn tại chu trình trong cấu trúc cây.

Nhờ đặc tính phân cấp rõ ràng, cấu trúc cây đặc biệt phù hợp để mô hình hoá các hệ thống có quan hệ thứ bậc như cây thư mục, sơ đồ tổ chức, và gia phả.

Trong bài toán quản lý gia phả, mỗi thành viên trong gia đình được biểu diễn bởi một nút trong cây, trong đó:

- Quan hệ cha-con tương ứng với quan hệ giữa các nút.
- Các thế hệ trong gia đình tương ứng với các mức (level) khác nhau của cây.
- Tổ tiên cao nhất được xem là nút gốc của cây gia phả.

Cách biểu diễn này cho phép mô hình hoá một cây nhiều nhánh với số lượng con bất kỳ, đồng thời đảm bảo cấu trúc dữ liệu gọn nhẹ, linh hoạt và thuận tiện cho việc cài đặt các thuật toán duyệt cây, thêm hoặc mở rộng gia phả trong ngôn ngữ C++.

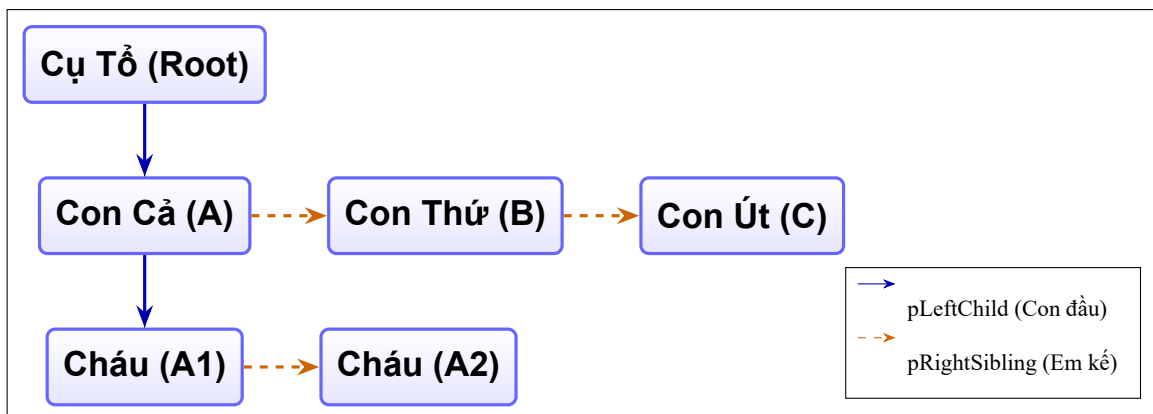
2.2.2. Biểu diễn cây bằng mô hình con đầu - anh em kế tiếp

Đối với cây nhiều nhánh, mỗi nút có thể có số lượng nút con không xác định. Nếu lưu trực tiếp danh sách con cho mỗi nút, cấu trúc dữ liệu sẽ trở nên phức tạp và khó quản lý. Để khắc phục vấn đề này, đề tài sử dụng mô hình con đầu-anh em kế tiếp (first-child / next-sibling) để biểu diễn cây gia phả.

Trong mô hình này, mỗi nút trong cây chỉ cần hai liên kết chính:

- Con trỏ `firstChild`: trỏ tới nút con đầu tiên của nút hiện tại.
- Con trỏ `nextSibling`: trỏ tới nút anh/chị/em kế tiếp có cùng nút cha.

Ngoài ra, mỗi nút còn lưu con trỏ `parent` trỏ tới nút cha, giúp việc truy vấn quan hệ cha - con và di chuyển ngược trong cây trở nên thuận tiện.



Hình 1: Hình minh hoạ mô hình biểu diễn cây theo cấu trúc con đầu - anh em kế tiếp

Với cách biểu diễn này:

- Nếu một nút không có con, con trỏ `firstChild` có giá trị `nullptr`.
- Nếu một nút là người con cuối cùng, con trỏ `nextSibling` có giá trị `nullptr`.
- Các con của cùng một nút cha được liên kết thành một danh sách liên kết đơn thông qua `nextSibling`.

Trong chương trình, mỗi thành viên trong gia phả được biểu diễn bởi một cấu trúc `Person`, trong đó các mối quan hệ được quản lý hoàn toàn bằng con trỏ. Việc thêm một người con mới được thực hiện bằng cách:

- Nếu người cha chưa có con, gán trực tiếp nút mới cho `firstChild`.
- Nếu người cha đã có con, duyệt danh sách các con thông qua `nextSibling` để tìm người con cuối cùng (con út), sau đó nối nút mới vào cuối danh sách.

Mã giả 1 Thêm con vào danh sách anh chị em

```

1: child ← TAOĐỐIƯỢNG(Person, name, gender, parent)
2: if parent.firstChild = null then                                     ☑ Nếu parent chưa có con đầu
3:     parent.firstChild ← child                                       ☑ Gán child làm con đầu tiên của parent
4: else
5:     current ← parent.firstChild
6:     while current.nextSibling ≠ null do
7:         current ← current.nextSibling
8:     current.nextSibling ← child                                     ☑ Nối child vào cuối danh sách con của parent

```

Hình 2: Mã giả mô tả thuật toán thêm một người con vào cây gia đình

Cách biểu diễn này mang lại các ưu điểm sau:

1. Biểu diễn được cây gia phả có số lượng con không giới hạn cho mỗi thành viên.
2. Cấu trúc dữ liệu gọn nhẹ, mỗi nút chỉ cần số lượng con trỏ cố định.
3. Thuận tiện cho việc duyệt cây theo chiều sâu và cài đặt các thuật toán xử lý quan hệ huyết thống.
4. Phù hợp với việc rèn luyện kỹ năng sử dụng con trỏ và quản lý bộ nhớ động trong C++.

Nhờ những đặc điểm trên, mô hình con đầu - anh em kế tiếp sử dụng con trỏ là lựa chọn phù hợp để hiện thực chương trình quản lý gia phả trong phạm vi đề tài.

2.2.3. Duyệt cây và quản lý bộ nhớ

Trong chương trình, việc xử lý và hiển thị gia phả được thực hiện thông qua duyệt cây theo chiều sâu, cụ thể là duyệt tiền thứ tự. Theo cách duyệt này, mỗi nút được xử lý trước, sau đó lần lượt duyệt đến nút con đầu tiên thông qua con trỏ `firstChild`, và tiếp tục duyệt các nút anh/chị/em kế tiếp thông qua con trỏ `nextSibling`.

Cách duyệt này rõ ràng phù hợp với mô hình con đầu - anh em kế tiếp và cho phép hiển thị gia phả theo thứ tự từ tổ tiên đến các thế hệ sau, phản ánh đúng cấu trúc phân cấp của gia đình.

Do chương trình sử dụng cấp phát bộ nhớ động cho các nút trong cây, việc quản lý và giải phóng bộ nhớ là yêu cầu bắt buộc. Khi kết thúc chương trình, toàn bộ các nút trong cây được giải phóng bằng một hàm đệ quy, đảm bảo không xảy ra rò rỉ bộ nhớ. Việc kết hợp duyệt cây và quản lý bộ nhớ động giúp chương trình hoạt động ổn định và an toàn trong môi trường C++.

```
1. // Depth-First Traversal
2. void DisplayTree(Person* current, int level) {
3.     if (current == nullptr) return;
4.     // Child
5.     DisplayTree(current->firstChild, level + 1);
6.     // Sibling
7.     DisplayTree(current->nextSibling, level);
8. }
9. // Free up memory
10. void FreeTree(Person* current) {
11.     if (current == nullptr) return;
12.     FreeTree(current->firstChild);
13.     FreeTree(current->nextSibling);
14.     delete current;
```

Hình 3: Đoạn mã C++ thể hiện logic duyệt cây theo chiều sâu và giải phóng bộ nhớ.

Đến đây, những cơ sở lý thuyết đã được trình bày ở trên đã căn bản làm nền tảng cho toàn bộ quá trình thiết kế chương trình sau này.

3. TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

3.1. Phát biểu bài toán

Trong việc quản lý gia phả của một dòng họ, thách thức lớn nhất nằm ở tính phân cấp và sự biến động của dữ liệu. Cụ thể, bài toán đặt ra các yêu cầu và ràng buộc sau:

- Thứ nhất, một cá nhân có thể là con của người này nhưng lại là cha/mẹ của người khác. Mỗi quan hệ này kéo dài qua nhiều thế hệ, tạo thành một cấu trúc cây.
- Thứ hai, khác với cây nhị phân (mỗi nút chỉ có tối đa 2 con), một người trong thực tế có thể có số lượng con bất kỳ. Việc sử dụng mảng cố định để lưu trữ danh sách con sẽ gây lãng phí bộ nhớ hoặc thiếu linh hoạt.
- Thứ ba, cần lưu trữ rõ ràng các thông tin thực thể (Tên, giới tính, ngày sinh...) và các mối quan hệ (Cha-con, Anh-chị-em).
- Cuối cùng, bài toán yêu cầu các thao tác: thêm mới, duyệt và tìm kiếm.

3.2. Cấu trúc dữ liệu

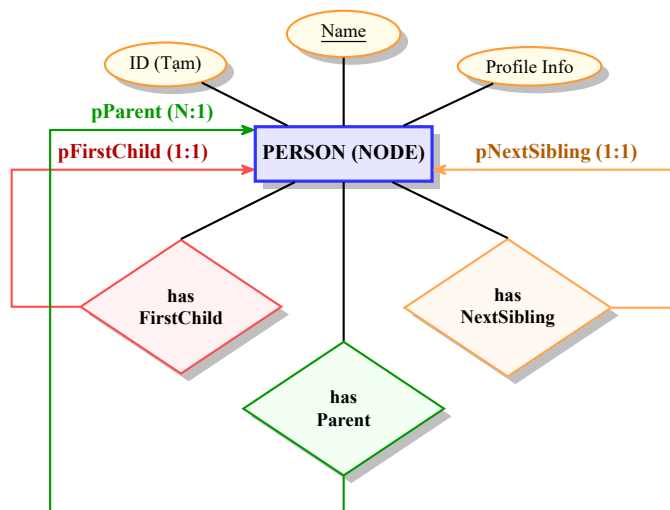
3.2.1. Thiết kế chung

Để giải quyết bài toán quản lý thông tin đa dạng của mỗi cá nhân trong dòng họ, chương trình thiết kế một kiểu dữ liệu tự định nghĩa (`struct person`). Sơ đồ cấu trúc được mô tả ở Hình 4. Cấu trúc này ánh xạ trực tiếp các yêu cầu của thực tế vào không gian bộ nhớ:

- Nhóm thuộc tính định danh và nhân khẩu học: Các trường dữ liệu `name`, `gender`, `birthday`, `job` được sử dụng để lưu trữ hồ sơ cơ bản. Trong đó, `gender` (Nam/Nữ) đóng vai trò là cờ điều hướng (flag) để quyết định cấu trúc phát triển của cây; trường `birthday` đóng vai trò định hình thứ tự anh - em trong những người con của cùng một cha.

- Nhóm thuộc tính gia đình và tình trạng: Trường `spouseName` (Tên vợ/chồng) được gán trực tiếp như một thuộc tính của Nút thay vì tạo một Nút độc lập, giúp giảm độ phức tạp của đồ thị cây. Ngoài ra, kỹ thuật này đảm bảo cấu trúc gia phả luôn là một cây đơn gốc, giúp duy trì tính toàn vẹn cho các thuật toán đệ quy và tìm tổ tiên chung (LCA) mà không làm tăng độ phức tạp của đồ thị thành dạng chu trình. Trường `deathDay` quản lý tình trạng sinh tử, và `numChildren` (số con) được thiết kế đặc biệt chỉ kích hoạt lưu trữ đối với thành viên Nữ.

- Nhóm thuộc tính định tuyến (Routing Attributes): Các con trỏ `pParent`, `pFirstChild`, `pNextSibling` đóng vai trò là “chỉ dẫn đường” để móc nối thực thể này vào mạng lưới gia phả chung, đảm bảo tính liên kết động mà không cần mảng trung gian.



Hình 4: Sơ đồ ERD mô tả thực thể Person và các mối quan hệ tự thân trong cấu trúc cây LCRS

3.2.2. Cài đặt các ràng buộc cần thiết

Cấu trúc dữ liệu đã được tùy biến để tuân thủ nghiêm ngặt các quy tắc truyền thống của một gia phả dòng họ, gồm hai ý chính:

1. Theo yêu cầu đề tài, con gái khi lập gia đình sẽ tính theo gia phả nhà chồng. Do đó, trong giải thuật `AddChild` (Thêm con), nhóm đã cài đặt chốt chặn logic, nghĩa là: Khi người dùng thực hiện thao tác thêm thế hệ mới, chương trình sẽ kiểm tra `parent->gender`. Nếu giá trị là “Nữ”, thao tác bị từ chối và nút đó vĩnh viễn là một “Nút lá” (không bao giờ có `pFirstChild`).

2. Trong một dòng họ, việc trùng họ tên giữa các thế hệ là khá phổ biến (ví dụ: nhiều người cùng tên “Nguyễn Văn A”). Việc tìm kiếm và cập nhật của chương trình nếu chỉ dựa vào chuỗi name sẽ dẫn đến sai lệch nghiêm trọng. Để giải quyết, nhóm cơ chế Lọc danh sách lựa chọn (Selection List). Thay vì định danh bằng chuỗi ký tự không duy nhất, chương trình sử dụng địa chỉ bộ nhớ làm định danh tuyệt đối, kết hợp với việc truy vấn thuộc tính phụ từ người dùng để xác nhận đối tượng.

3.3. Thuật toán và đánh giá thuật toán

3.3.1. Thuật toán Duyệt và Tìm kiếm

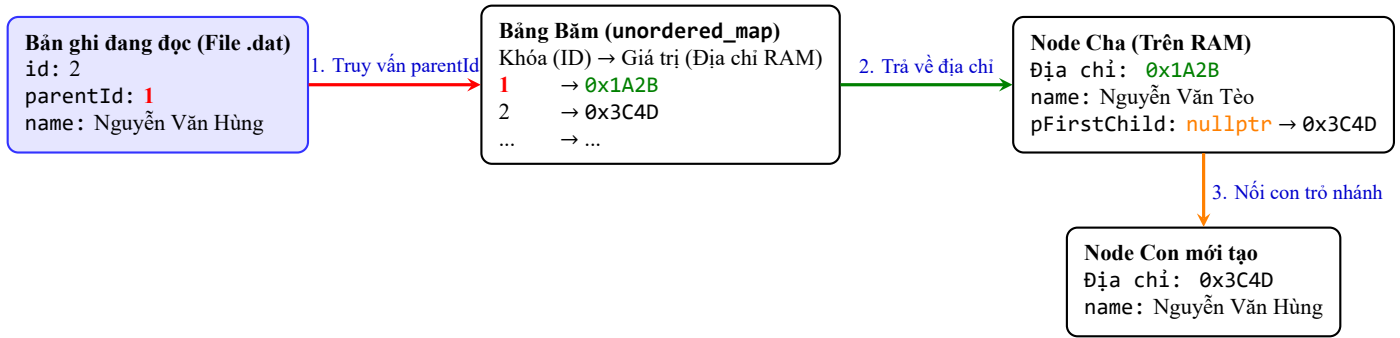
Vì các nút bộ nhớ (`Person`) được cấp phát động trên RAM, chương trình thiết kế hai chiến lược truy vấn chuyên biệt dựa trên loại khóa tìm kiếm để tối ưu hóa hiệu năng hệ thống:

Truy vấn theo Định danh (ID Lookup)

- *Cơ sở lý thuyết:* Sử dụng cấu trúc Bảng băm (Hash Table) nhằm thiết lập ánh xạ trực tiếp $f : ID \rightarrow \text{Memory Address}$, cho phép truy xuất dữ liệu độc lập với độ sâu của đồ thị cây.

- *Phương pháp thực hiện:* Kỹ thuật này được kích hoạt trong tiến trình đọc tệp nhị phân `.dat`. Chương trình khởi tạo `std::unordered_map<int, Person*>` để lưu trữ danh sách các nút vừa cấp phát. Khi cần khôi phục tập hợp cạnh E (nổi con trở cha-con), hệ thống chỉ cần truyền `parentId` vào Bảng băm để nhận lại con trở thực thi ngay lập tức. Hình 5 mô tả dễ hiểu quá trình này.

- *Phân tích độ phức tạp:* Thuật toán đưa độ phức tạp thời gian tìm kiếm cha từ $O(|V|)$ xuống $O(1)$ tuyệt đối, giải quyết triệt để nút thắt cổ chai khi hệ thống phải tái thiết lập hàng vạn liên kết con trở.



Hình 5: Mô phỏng quy trình thực hiện truy vấn theo Định danh bằng cấu trúc Bảng băm

Pha 1: Cấp phát nút	Pha 2: Thiết lập liên kết
Khởi tạo bảng băm: <code>HashMap<int, Person*></code> for mỗi Record trong fileRecords do pNew = New Person(Record.name, Record.gender) Sao chép thông tin từ Record sang pNew <code>HashMap[Record.id] = pNew</code>	for mỗi Record trong fileRecords do currentPerson = <code>HashMap[Record.id]</code> if Record.parentId != 0 then Tra cứu địa chỉ Cha từ Bảng băm Thực hiện nối nút Cập nhật <code>global_id_counter = Max(Record.id) + 1</code> Trả về địa chỉ Gốc (Root)

Hình 6: Mã giả quy trình thực hiện truy vấn theo Định danh bằng cấu trúc Bảng băm

Truy vấn theo Tên gọi

- *Cơ sở lý thuyết:* Do thuộc tính “Tên” là dữ liệu chuỗi phi tuyến tính (không tạo thành cây tìm kiếm nhị phân BST), hệ thống triển khai thuật toán Duyệt theo chiều sâu (Depth-First Search - DFS).

- *Phương pháp thực hiện:* Thuật toán sử dụng đệ quy Tiền thứ tự (Pre-order), duyệt tuần tự qua các liên kết pFirstChild và pNextSibling. Để khắc phục điểm yếu của thuật toán tham lam (Greedy) truyền thống, chương trình không trả về kết quả khớp đầu tiên. Thay vào đó, thuật toán tiếp tục duyệt cạn không gian đỉnh để thu thập toàn bộ các thực thể trùng tên vào một `std::vector<Person*>`. Kết quả này sau đó được đẩy lên tầng giao diện (UI) để người dùng phân biệt và chọn lựa mục tiêu chính xác.

- *Phân tích độ phức tạp:* Độ phức tạp thời gian là $O(|V|)$ (số lượng thành viên). Độ phức tạp không gian lưu trữ cho ngăn xếp gọi hàm (Call Stack) là $O(H)$, với H là chiều cao cực đại của cây gia phả. Với quy mô dữ liệu thực tế của một dòng họ, thuật toán đảm bảo tốc độ phản hồi ở mức thời gian thực.

3.3.2. Thuật toán Xác định quan hệ

Bài toán Xác định quan hệ và danh xưng trong gia phả thực chất là một bài toán kinh điển của cấu trúc dữ liệu cây. Dưới góc độ cơ sở toán học trong khoa học máy tính, đây chính

là sự kết hợp giữa thuật toán Tìm tổ tiên chung gần nhất (Lowest Common Ancestor - viết tắt là LCA) và việc phân tích Ma trận độ lệch khoảng cách.

Để đảm bảo thuật toán chạy chính xác, cũng như bao quát được sự phức tạp của hệ thống danh xưng trong tiếng Việt, thuật toán sẽ trải qua bốn pha như sau:

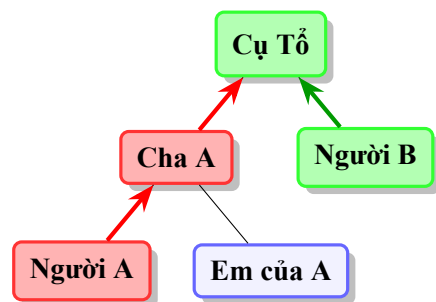
Pha thứ nhất: Truy vết đường đi về gốc.

- Mục tiêu của pha: Tìm chuỗi các đỉnh kết nối từ người đang xét lên tới Cụ Tổ, làm cơ sở cho việc xác định LCA ở pha thứ hai.
- Phương pháp thực hiện: Giả sử đồ thị cây gia phả là $T = (V, E)$, ta cần xây dựng hai tập hợp đường đi cho nút A và nút B . Quy trình thực hiện như sau:
 - Từ con trỏ của A , dùng vòng lặp di chuyển theo hướng con trỏ `pParent` cho đến khi gặp `nullptr` (Nút gốc).
 - Lưu lại các địa chỉ bộ nhớ đã đi qua vào một mảng (hoặc `std::vector`). Ta thu được tập $P_A = \{A, P_{A1}, P_{A2}, \dots, \text{Root}\}$.
 - Làm tương tự với B , ta thu được tập $P_B = \{B, P_{B1}, P_{B2}, \dots, \text{Root}\}$.
 - Về độ phức tạp: Độ phức tạp thời gian xấp xỉ $O(H)$, với H là chiều cao lớn nhất của cây (số lượng thế hệ). Thao tác này cực kỳ nhanh vì nó chỉ đi theo một đường thẳng lên trên, không cần duyệt qua các nhánh khác của gia phả; Độ phức tạp không gian là $O(H)$ vì cần cấp phát hai mảng để lưu trữ tối đa H con trỏ địa chỉ bộ nhớ cho mỗi đường đi.

Mã giả 2 GetPathToRoot(Node)

```

Khởi tạo mảng Path rỗng
while Node không phải là rỗng do
    Thêm Node vào đầu mảng Path (để Gốc luôn ở vị trí số 0)
    Node = Node.parent
return Path
  
```



Hình 7: Mã giả và hình vẽ minh họa cho pha 1 của thuật toán.

Pha thứ hai: Xác định Tổ tiên chung gần nhất (LCA).

- Mục tiêu của pha: Tìm ra nút chung đầu tiên mà cả hai đối tượng A và B cùng thuộc về nhánh hậu duệ của nút đó.
- Phương pháp thực hiện: Ta thực hiện phép so sánh hai tập P_A và P_B đã có được từ pha thứ nhất, cụ thể:
 - So sánh hai danh sách P_A và P_B từ vị trí nút gốc (đầu danh sách).

– Nút cuối cùng trùng khớp trong cả hai danh sách trước khi đường đi rẽ nhánh chính là LCA cần tìm.

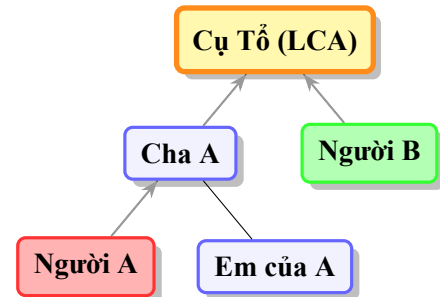
• Về độ phức tạp: Độ phức tạp thời gian là $O(K)$, với $K = \min(h_A, h_B)$, phép so sánh được thực hiện trực tiếp trên địa chỉ bộ nhớ thay vì so sánh chuỗi.

Mã giả 3 DetermineRelationship (A, B)

```

Đặt PathA = GetPathToRoot(A)
Đặt PathB = GetPathToRoot(B)
for i from 0 to min(hA, hB) do
    if PathAi trùng PathBi then
        Đặt LCAIndex = i
    else break
if LCAIndex = -1 then
    return Không cùng huyết thống
else return LCAIndex

```



Hình 8: Mã giả và hình vẽ minh họa cho pha 2 của thuật toán

Pha thứ ba: Tính toán véc-tơ khoảng cách.

• Mục tiêu của pha: Định lượng khoảng cách thế hệ giữa các đối tượng và tổ tiên chung để làm cơ sở cho việc phân giải danh xưng.

• Phương pháp thực hiện: Gọi d_A là khoảng cách thế hệ từ LCA đến A. Gọi d_B là khoảng cách thế hệ từ LCA đến B. Lúc này, ta có một cặp tọa độ (d_A, d_B) phản ánh cấp bậc của hai người, làm tiền đề cho pha cuối cùng.

• Về độ phức tạp: Vì mảng đường đi chứa tuần tự các đỉnh từ Gốc đến nút đích, khoảng cách d được chứng minh bằng công thức toán học tuyệt đối: $d = (M - 1) - T$ với M là chiều dài mảng và T là vị trí của LCA. Vậy độ phức tạp thời gian và không gian cho pha này chỉ là $O(1)$.

Pha cuối cùng: Phân giải danh xưng

• Mục tiêu của pha: Chuyển đổi các giá trị toán học (d_A, d_B) thành tên gọi quan hệ họ hàng theo quy chuẩn văn hóa.

• Phương pháp thực hiện: Sử dụng cấu trúc điều kiện để ánh xạ cặp (d_A, d_B) vào ma trận danh xưng, cụ thể:

Tọa độ (d_A, d_B)	Quan hệ của B đối với A	Quan hệ của A đối với B	Giải thích Logic
($k, 0$)	Tổ tiên đời thứ k	Hậu duệ đời thứ k	$B \equiv L$, B là gốc của nhánh chứa A.

Còn tiếp

Tọa độ (dA, dB)	Quan hệ của B đối với A	Quan hệ của A đối với B	Giải thích Logic
(0, k)	Hậu duệ đời thứ k	Tổ tiên đời thứ k	$A \equiv L$, A là gốc của nhánh chứa B .
(k, k)	Anh/Chị/Em họ đời $k - 1$	Anh/Chị/Em họ đời $k - 1$	Chung tổ tiên cách đây k thế hệ.
(1, 2)	Bác/Cô/Chú/Dì	Cháu (gọi bằng Bác/Chú...)	B là con của anh/chị/em với A .
(2, 1)	Cháu (gọi bằng Bác/Chú...)	Bác/Cô/Chú/Dì	A là con của anh/chị/em với B .
(1, k) với $k > 2$	Cụ/Ký (nhánh bàng hệ)	Chắt/Chút (nhánh bàng hệ)	B là hậu duệ xa của anh/chị/em với A .
(dA, dB)	Họ hàng tổng quát	Họ hàng tổng quát	Cách nhau tổng $dA + dB$ bậc thế hệ.

3.3.3. Thuật toán truy vấn theo danh xưng

Một bài toán khác được đặt ra ngược với bài toán được xác định ở Mục 3.3.2, đó là tìm người theo quan hệ, nghĩa là: cho một người xác định và quan hệ (bố mẹ/ông bà/cô chú/...), ta cần phải tìm được một (hoặc nhiều) người có quan hệ đó với người đã xác định. Hệ thống tận dụng tính đối ngẫu của thuật toán LCA trước đó. Thay vì tìm tọa độ từ hai nút, ta sẽ cố định tọa độ (d_A, d_B) dựa trên danh xưng tiếng Việt và thực hiện truy vết ngược từ con trở pParent kết hợp duyệt nhánh ngang pNextSibling để trả về kết quả cần thiết. Cụ thể, thuật toán này trải qua ba pha như sau:

Pha thứ nhất: Xác định tổ tiên.

- Mục tiêu của pha: Tìm ra nút tổ tiên chung (L).
- Phương pháp thực hiện: Từ nút A , di chuyển qua con trở pParent đúng d_A lần. Ví dụ: Tìm “Chú” thì $d_A = 2$. Từ A , ta cần leo lên “Bố” ($d = 1$), rồi sau đó lên “Ông nội” ($d = 2$). Nút Ông nội chính là L .
- Về độ phức tạp: Độ phức tạp thời gian là $O(d_A)$, vì chỉ đi theo một đường thẳng nên rất nhanh.

Pha thứ hai: Duyệt và lọc đối tượng.

- Mục tiêu của pha: Thu thập được những người phù hợp.
- Phương pháp thực hiện: Từ tổ tiên L , ta cần tìm các hậu duệ ở mức thế hệ d_B thỏa mãn các điều kiện về giới tính và thứ tự. Ta sẽ sử dụng phương pháp duyệt theo chiều

Mã giả 4 FindAncestor(startP, dA)

```

1: temp ← startP
2: for i = 1 → dA do
3:   if temp.parent ≠ NULL then
4:     temp ← temp.parent
5:   else return NULL
6: return temp

```

⌚ Không đủ độ sâu thế hệ

Hình 9: Mã giả mô tả pha thứ nhất của thuật toán truy vấn theo danh xưng

rộng (BFS) để tìm tất cả các nút cách L đúng d_B bậc.

- Về độ phức tạp: Độ phức tạp thời gian là $O(S)$, với S là số lượng anh/em/con/cháu trong nhánh đang xét đó.

Mã giả 5 Thuật toán thu thập ứng viên cùng thế hệ

```

1: function COLLECTCANDIDATES(root, dB)
2:   Cands ← ∅
3:   if dB = 0 then
4:     return {root}
5:   procedure DUYETANG(curr, depth)
6:     if curr = NULL then
7:       return
8:     if depth = dB then
9:       Cands ← Cands ∪ {curr}
10:      return
11:     DUYETANG(curr.firstChild, depth + 1)
12:     t ← curr.firstChild
13:     while t ≠ NULL and t.nextSibling ≠ NULL do
14:       t ← t.nextSibling
15:       DUYETANG(t, depth + 1)
16:   DUYETANG(root, 0)
17:   return Cands

```

⌚ Khởi tạo tập ứng viên rỗng

⌚ Nếu khoảng cách = 0, trả về chính gốc

⌚ Trường hợp cơ sở: không có node để xử lý

⌚ Thêm node hiện tại vào tập ứng viên

⌚ Dừng duyệt sâu hơn

⌚ Duyệt node con đầu tiên (xuống sâu hơn)

⌚ Bắt đầu duyệt các node anh em

⌚ Chuyển sang node anh em kế tiếp

⌚ Duyệt cây con của node anh em này

⌚ Bắt đầu duyệt từ gốc ở độ sâu 0

⌚ Trả về tất cả các ứng viên đã thu thập được

Hình 10: Mã giả mô tả pha thứ hai của thuật toán truy vấn theo danh xưng**Pha thứ ba: Phân giải danh xưng cụ thể.**

- Mục tiêu của pha: Áp dụng những ràng buộc đã có để đưa ra người cần tìm kiếm.
- Phương pháp thực hiện: Đây là lúc áp dụng các ràng buộc mà nhóm đã thiết kế trong struct **Person**, cụ thể là phân giải theo “Giới tính” và “Thứ tự”. Cụ thể:
 - Theo Giới tính: Nếu yêu cầu là “Cô”, lọc những người có gender là “Nữ”.
 - Theo Thứ tự: Dựa vào con trỏ **pNextSibling**. Nếu người đó nằm phía trước Cha của A trong danh sách anh em, đó là “Bác”, ngược lại là “Chú”.

Tổng quát, thuật toán chung của ta vẫn tiệm cận mức $O(H)$, rõ ràng là hiệu quả hơn nhiều so với việc quét toàn bộ người trong gia phả. Ngoài ra, thuật toán này còn có ưu điểm là

xử lý được trường hợp đa kết quả (ví dụ: tìm “Chú” có thể cho ra nhiều người, việc tìm người cụ thể do người dùng quyết định).

Ngoài ra, vì cấu trúc LCRS truyền thống chỉ lưu trữ các thực thể có quan hệ huyết thống trực tiếp, khiến các thành viên nữ (vợ) trở thành “điểm mù” (nghĩa là, không thể thực hiện bất kỳ chức năng nào của hệ thống trên những người này) của giải thuật duyệt cây. Vì vậy, giải pháp cho điều này là sử dụng cơ chế Proxy Node (Nút đại diện). Khi truy vấn “Mẹ”, hệ thống thực hiện tìm kiếm gián tiếp: $Mother(A) = Spouse(Parent(A))$, cụ thể:

- Hệ thống xác định nút Cha của A (là nút P trong cây).
- Sau đó, hệ thống truy xuất vào thuộc tính spouseName (vợ) của nút P.

Cách làm này giữ cho cây luôn là cây đơn gốc (single-root tree), giúp các thuật toán duyệt đệ quy và LCA luôn đạt độ phức tạp tối ưu nhất.

3.3.4. Thuật toán lưu trữ và phục hồi

Trong môi trường quản lý dữ liệu bằng con trỏ động, địa chỉ bộ nhớ của các thực thể (Person) chỉ có giá trị nội hàm trong phiên làm việc hiện tại của RAM. Để đảm bảo tính bền vững của dữ liệu, nhóm áp dụng kỹ thuật Tuần tự hóa nhị phân (Binary Serialization) kết hợp với thuật toán Phẳng hóa đồ thị cây.

Cấu trúc Bản ghi tĩnh trung gian (PersonRecord)

Để khắc phục nhược điểm của chuỗi động `std::string`, chương trình định nghĩa một cấu trúc PersonRecord kích thước cố định, sử dụng mảng ký tự tĩnh (`char[ ]`). Các liên kết con trỏ LCRS (`pParent`, `pFirstChild`, `pNextSibling`) được thay thế hoàn toàn bằng định danh số nguyên là `id` (mã cá nhân) và `parentId` (mã của nút cha).

Giải thuật Ghi tệp (Phẳng hóa cây LCRS)

- Hệ thống sử dụng thuật toán duyệt cây Tiên thứ tự (Pre-order Traversal) thông qua hàm đệ quy `FlattenTree`.
- Tại mỗi nút, dữ liệu được sao chép an toàn (sử dụng `strncpy` để chống tràn bộ đệm) sang cấu trúc PersonRecord và đẩy vào một mảng động `std::vector`.
- Thay vì ghi tuần tự từng cá nhân, chương trình áp dụng kỹ thuật Block I/O: tính toán tổng dung lượng của mảng và sử dụng phương thức `write()` để đổ toàn bộ khối dữ liệu từ RAM xuống tệp `.dat` trong một chu kỳ ghi duy nhất.

Mã giả 6 Hàm FlattenTree**Require:** currentNode, recordArray

```

1:                                                                 ❶ Điều kiện dừng đệ quy
2: if currentNode là RÕNG then return
3:                                                                 ❷ 1. Chuyển đổi Node động thành Bản ghi tĩnh
4:   Tạo NewRecord
5:   NewRecord.id = currentNode.id
6:   if currentNode có Cha then
7:     NewRecord.parentId = currentNode.parent.id
8:   else
9:     NewRecord.parentId = 0                                                                 ❷ 0 ám chỉ đây là Cụ Tổ (Gốc)
10:  Chép các thông tin (Tên, Năm sinh...) từ Node sang Bản ghi
11:                                                                 ❷ 2. Lưu vào mảng
12:  Thêm NewRecord vào recordArray
13:                                                                 ❷ 3. Đệ quy đi tiếp xuống nhánh Con và nhánh Em
14:  FlattenTree(currentNode.firstChild, recordArray)
15:  FlattenTree(currentNode.nextSibling, recordArray)

```

Hình 11: Mã giả mô tả thuật toán phẳng hoá cây để lưu file***Giải thuật Đọc tệp và Tái cấu trúc***

Quá trình phục hồi dữ liệu từ tệp nhị phân được tối ưu hóa ở mức cao nhất nhằm tránh việc tìm kiếm lặp lại:

1. Chương trình đọc toàn bộ khối dữ liệu từ tệp lên một mảng PersonRecord trên RAM.
2. Khởi tạo một Bảng băm (Hash Map) sử dụng cấu trúc `std::unordered_map<int, Person*>` để thiết lập quan hệ ánh xạ giữa `id` nguyên thủy và địa chỉ bộ nhớ động vừa được cấp phát.
3. Khi duyệt qua từng bản ghi, hệ thống tạo mới một đối tượng `Person` và sử dụng `parentId` để tra cứu trực tiếp địa chỉ người cha trong Bảng băm. Thao tác tra cứu này đạt độ phức tạp thời gian $O(1)$, cho phép liên kết con trở ngay lập tức thông qua hàm `AddChild` mà không cần duyệt lại cây LCRS.
4. Hệ thống tự động cập nhật lại biến đếm ID toàn cục (`global_id_counter`) dựa trên mức ID lớn nhất trong tệp để đảm bảo tính duy nhất cho các phiên nhập liệu tiếp theo.

Đánh giá thuật toán

Cả hai tiến trình `Serialization` và `Deserialization` đều đạt độ phức tạp thời gian $O(N)$ và không gian $O(N)$, với N là tổng số thành viên trong gia phả. Việc kết hợp `Block I/O` và `Hash Map` giúp giải thuật xử lý mượt mà dữ liệu quy mô hàng vạn thực thể mà không

gây ra tình trạng tràn ngăn xếp (Stack Overflow) hay nghẽn cổ chai truy xuất đĩa.

3.3.5. Thuật toán thống kê dòng họ

Thuật toán thống kê cung cấp cái nhìn tổng quan về quy mô và cấu trúc của dòng họ thông qua các chỉ số định lượng, nhằm giúp người quản trị nắm bắt được tổng số thành viên (bao gồm cả huyết thống và phu nhân), sự phân bố giới tính, độ dài thế hệ (số đời), và tình trạng sinh tồn của các cá nhân trong gia phả.

Do gia phả được lưu trữ dưới dạng cây LCRS, việc thống kê được thực hiện tối ưu nhất bằng phương pháp Duyệt cây theo chiều sâu (DFS) kết hợp với kỹ thuật đệ quy. Mỗi khi đi qua một nút (*Node*), hệ thống sẽ thực hiện kiểm tra các thuộc tính của thực thể *Person* và cập nhật vào cấu trúc dữ liệu *FamilyStats*.

Các bước cụ thể như sau:

- Bước 1: Tạo một cấu trúc lưu trữ *FamilyStats* với các biến đếm (tổng số, nam, nữ, phu nhân, tử vong) và một mảng *genDistribution* để lưu số lượng thành viên của từng đời.
- Bước 2: Đệ quy xử lý nút hiện tại. Tăng biến *totalMembers*; Kiểm tra *gender*: Nếu là “Nam” tăng *maleCount*, nếu “Nu” tăng *femaleCount*; kiểm tra *spouseName*: Nếu khác “None” hoặc “N/A”, tăng *totalSpouses* (thống kê dâu/rể); Kiểm tra *deathDay*: Nếu khác “N/A”, tăng *deceasedCount*; Cập nhật thế hệ: Sử dụng tham số *level* để tăng giá trị tại *genDistribution[level]* và cập nhật $\text{maxGen} = \max(\text{maxGen}, \text{level})$.
- Bước 3: Lan toả đệ quy. Gọi đệ quy xuống con đầu tiên và gọi đệ quy sang anh em kế cận.

Phân tích độ phức tạp

- Độ phức tạp thời gian: $O(N)$, với N là tổng số nút trong cây gia phả. Thuật toán chỉ đi qua mỗi nút đúng một lần duy nhất.
- Độ phức tạp không gian: $O(H)$, với H là chiều cao của cây (tương ứng với số đời), do sử dụng ngăn xếp đệ quy trong quá trình duyệt.

4. CHƯƠNG TRÌNH VÀ KẾT QUẢ

4.1. Tổ chức chương trình

4.1.1. Kiến trúc tổng thể của chương trình

Hệ thống được thiết kế và xây dựng dựa trên mô hình Lập trình theo mô-đun (module). Thay vì tập trung toàn bộ mã nguồn vào một tệp duy nhất, chương trình được phân rã thành các mô-đun độc lập dựa trên *nguyên lý Phân tách trách nhiệm*.

Cách tiếp cận này đảm bảo hệ thống đạt được độ gắn kết cao bên trong từng mô-đun và độ phụ thuộc thấp giữa các mô-đun với nhau, tạo tiền đề thuận lợi cho việc kiểm thử, gỡ lỗi và mở rộng tính năng trong tương lai.

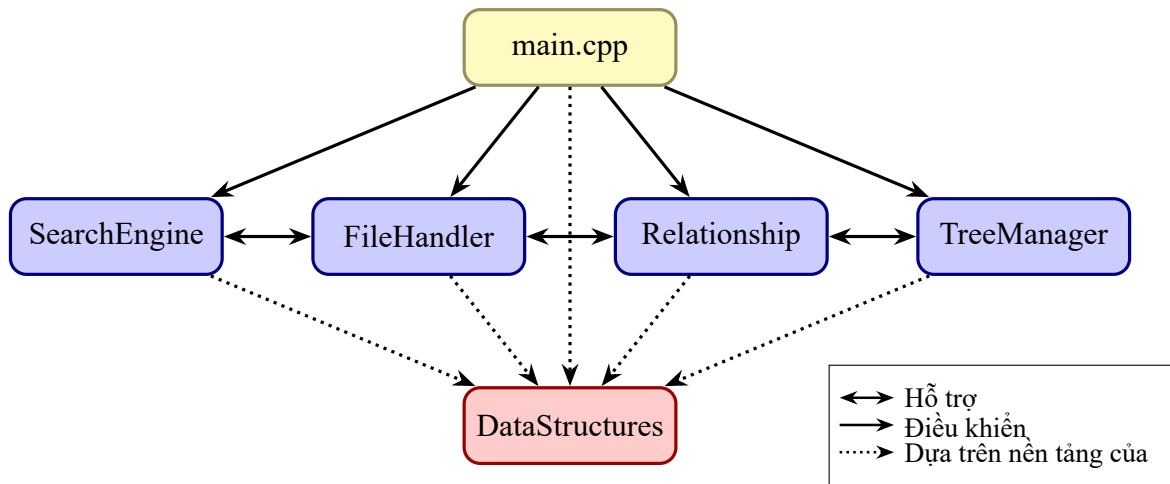
4.1.2. Cấu trúc tệp tin và phân rã chức năng

Mã nguồn của đồ án được tổ chức thành 6 thành phần cốt lõi, bao gồm 1 tệp tiêu đề cơ sở, 4 cặp mô-đun logic (.h và .cpp) và một tệp điều khiển trung tâm. Chi tiết chức năng của từng tệp được quy định như sau:

- (a) `DataStructures.h`: Định nghĩa cấu trúc nút LCRS, bản ghi và biến toàn cục, đảm bảo nhất quán kiểu dữ liệu giữa các mô-đun.
- (b) `TreeManager.h/.cpp`: Thao tác trực tiếp trên RAM; khởi tạo biến toàn cục; quản lý vòng đời nút (cấp phát, giải phóng, truy xuất).
- (c) `SearchEngine.h/.cpp`: Xử lý truy vấn, kiểm soát trùng lặp; duyệt DFS để tìm kiếm; cung cấp giao diện chọn thực thể.
- (d) `Relationship.h/.cpp`: Giải quyết bài toán đồ thị gia phả; tìm tổ tiên chung (LCA); đổi tọa độ thế hệ sang danh xưng và ngược lại.
- (e) `FileHandler.h/.cpp`: Quản lý I/O với ổ cứng; tuần tự hóa cây LCRS thành mảng, ghi/đọc file nhị phân; hỗ trợ nhập từ file văn bản, xử lý lỗi dòng.
- (f) `main.cpp`: Khởi chạy, tạo dữ liệu mẫu, điều hướng sự kiện và xử lý lỗi nhập.

Toàn bộ mã nguồn của chương trình quản lý gia phả có tại Phụ lục A đính kèm theo báo cáo này.

Sơ đồ cấu trúc các hàm chính của chương trình có tại Phụ lục B đính kèm theo báo cáo này.



Hình 12: Sơ đồ mô tả cấu trúc chương trình và các tệp tin

4.2. Ngôn ngữ cài đặt

4.2.1. Lý do sử dụng ngôn ngữ C++

Dự án lựa chọn ngôn ngữ C++ làm ngôn ngữ lập trình chính để triển khai hệ thống bởi những lý do như sau:

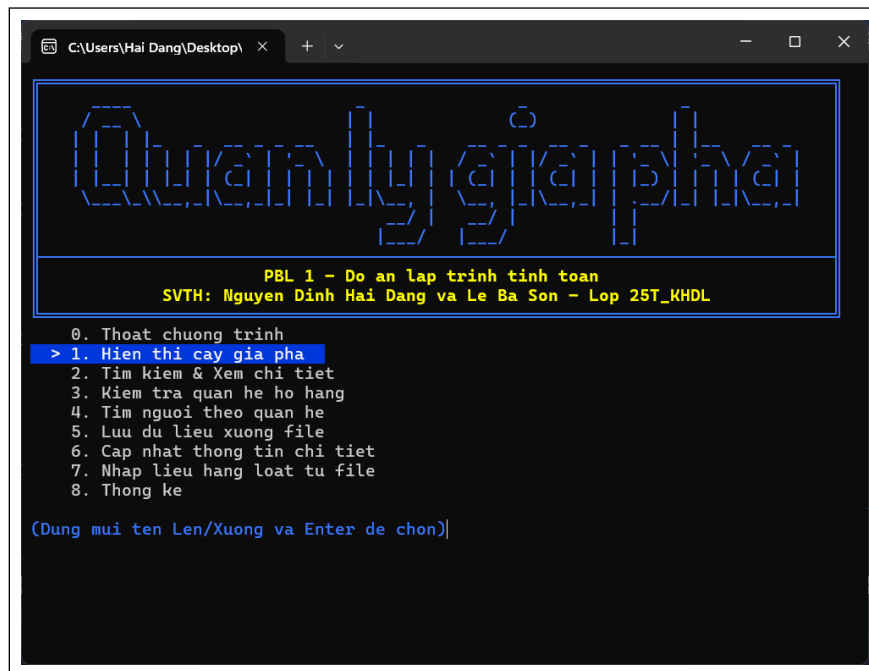
- Mô hình LCRS yêu cầu cấp phát bộ nhớ liên tục, C++ dùng new/delete để kiểm soát vòng đời của từng nút (Person) một cách chính xác, nhờ đó tối ưu hiệu suất.
- Dùng `std::string` thay vì mảng char giúp xử lý tên tiếng Việt an toàn, tránh tràn bộ đệm. `std::vector` cũng được dùng để lưu kết quả tìm kiếm, giúp mã nguồn ngắn gọn, rõ ràng.
- Nhờ struct/class, C++ mô hình hóa “Con người” tự nhiên với đầy đủ thuộc tính và quan hệ, đồng thời dễ dàng tách chương trình thành các tệp .h và .cpp.

4.2.2. Môi trường phát triển và công cụ hỗ trợ

- Trình biên dịch: Sử dụng GCC (g++) tiêu chuẩn C++11 trở lên để hỗ trợ các tính năng hiện đại và đảm bảo tốc độ thực thi tối ưu.
- Hệ thống được phát triển trên các IDE phổ biến như *Embarcadero Dev-C++* và *Visual Studio*, tận dụng khả năng quản lý project để liên kết các tệp tin mô-đun.
- Định dạng tệp lưu trữ: tệp nhị phân (.dat) dùng để lưu trữ trạng thái cây dưới dạng các struct tĩnh, đảm bảo tốc độ đọc/ghi nhanh và bảo mật dữ liệu cơ bản; tệp văn bản (.txt) dùng để nhập liệu hàng loạt theo định dạng phân tách bằng dấu phẩy (CSV), giúp người dùng dễ dàng soạn thảo dữ liệu từ bên ngoài.

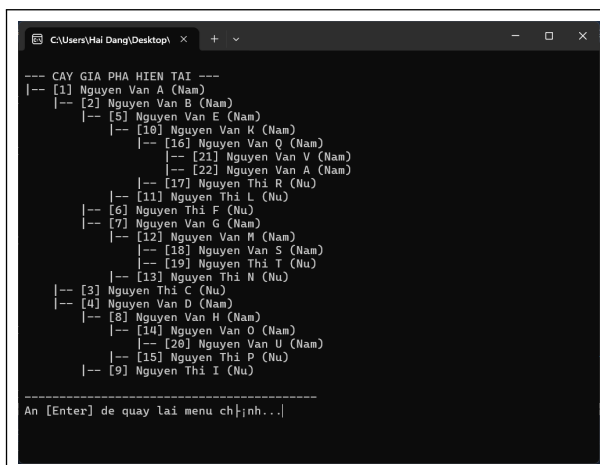
4.3. Kết quả

4.3.1. Giao diện chính của chương trình

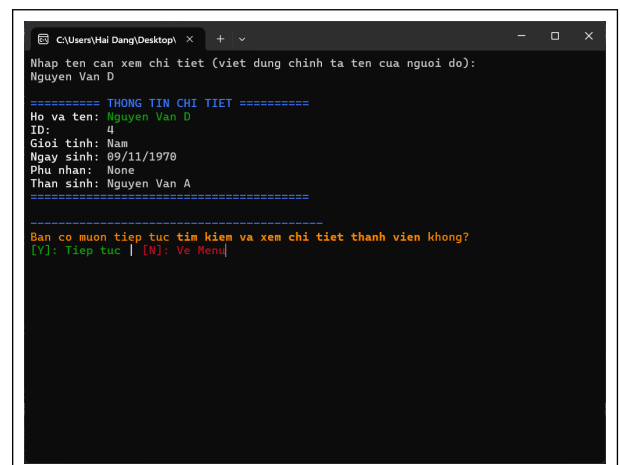


Hình 13: Giao diện chính của chương trình

4.3.2. Kết quả thực thi chương trình



(a) Hiện thị cây gia phả



(b) Tìm kiếm người trong gia phả

Hình 14: Tổng hợp các màn hình thực thi các chức năng chính của hệ thống quản lý gia phả


```

C:\Users\Hai Dang\Desktop\ x + v
Nhập tên người thu nhất (viết đúng chính tả tên của người do):
Nguyen Van D
Nhập tên người thu hai (viết đúng chính tả tên của người do):
Nguyen Van H

==> KẾT QUẢ: Nguyen Van D là Cha của Nguyen Van H

-----
Bạn có muốn tiếp tục kiểm tra quan hệ họ hàng không?
[Y]: Tiếp tục | [N]: Về Menu

```

(c) Kiểm tra quan hệ họ hàng giữa hai người

```

C:\Users\Hai Dang\Desktop\ x + v
--- TÌM NGƯỜI THEO QUAN HỆ ---
Nhập tên người làm mốc (viết đúng chính tả tên của người do):
Nguyen Van H

===== CHON QUAN HE CAN TIM =====
1. Cha      2. Mẹ      3. Ông      4. Bà
5. Anh trai 6. Chị gái  7. Em trai 8. Em gái
9. Bác trai 10. Bác gái 11. Chú     12. Cô
14. Anh/Chị/Em họ 15. Con     16. Cháu
17. Cháu/Chat/... (Hậu duệ của Anh/Chị/Em)

Nhập lựa chọn (1-17): 14

=> Tìm thấy 4 Anh/Chị/Em họ của Nguyen Van H:

- Nguyen Van E
- Nguyen Thi F
- Nguyen Van G
- Nguyen Thi I

-----
Bạn có muốn tiếp tục tìm người theo quan hệ không?
[Y]: Tiếp tục | [N]: Về Menu

```

(d) Tìm người theo quan hệ

```

C:\Users\Hai Dang\Desktop\ x + v
Nhập tên người cần cập nhật (viết đúng chính tả tên của người do):
Nguyen Van G

--- Cập nhật thông tin cho: [Y] Nguyen Van G ---
(Nhấn Enter nếu muốn giữ nguyên thông tin cũ)
Ngày sinh hiện tại [05/09/1995]: 06/09/1995
Nghề nghiệp hiện tại [Unknown]: Giáo viên
Ngày mất hiện tại [N/A]: N/A
Tên vợ/chồng hiện tại [None]: Nguyen Thi Sinh

[OK] Cập nhật thành công!

--- THÔNG TIN SAU KHI CẬP NHẬT ---
===== THÔNG TIN CHI TIẾT =====
Họ và tên: Nguyen Van G
ID: 7
Giới tính: Nam
Ngày sinh: 06/09/1995
Phụ nhân: Nguyen Thi Sinh
Thân sinh: Nguyen Van B
=====

Bạn có muốn tiếp tục cập nhật thông tin không?
[Y]: Tiếp tục | [N]: Về Menu

```

(e) Cập nhật thông tin cho cá nhân

```

C:\Users\Hai Dang\Desktop\ x + v
===== THỐNG KÊ GIA PHẢ =====
1. Quy mô dòng họ: 23 người
   - Huyết thống: 22
   - Phụ nhân: 1
2. Số đời (Generations): 6
3. Cơ cấu giới tính: Nam: 14 | Nữ: 8
4. Tình trạng: Con sống: 22 | Đã khuất: 0

--- biểu đồ phân bố thành viên theo đời ---
Đời 01: * (1)
Đời 02: *** (3)
Đời 03: ***** (5)
Đời 04: ***** (6)
Đời 05: ***** (5)
Đời 06: ** (2)

=====
Nhấn Enter để quay lại menu...

```

(f) Thống kê gia phả

```

C:\Users\Hai Dang\Desktop\ x + v
===== NẠP DỮ LIỆU GIA PHẢ =====
1. Nạp từ file nhị phân (.dat) - Phục hồi trạng thái cũ
2. Nạp từ file văn bản (.txt) - Nhập mới hàng loạt
0. Quay lại Menu chính

Nhập lựa chọn của bạn: 2

[Lưu ý] File .txt phải đúng định dạng: Cha, Con, Giới Tính, Nam Sinh, Ngày Sinh
Nhập tên file văn bản (VD: NhapLieu.txt): SampleFamily2.txt
[Canh báo] Gia phả hiện tại sẽ bị xóa sạch để nạp dữ liệu mới.
Bạn có chắc chắn muốn tiếp tục? (y/n): y
[OK] Đã xóa sạch toàn bộ dữ liệu gia phả trên RAM!
ID hệ thống đã được reset về 1.

[OK] Đã đọc và sắp xếp 22 thành viên từ file text theo thu từ nam sinh!

An [Enter] để tiếp tục...

Bạn có muốn tiếp tục thao tác nạp file không?
[Y]: Tiếp tục | [N]: Về Menu

```

(g) Nhập gia phả từ file .txt

Hình 14: Tổng hợp các màn hình thực thi các chức năng chính của hệ thống quản lý gia phả (tiếp theo)

5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Sau quá trình nghiên cứu, thiết kế và cài đặt hệ thống “Quản lý Gia phả”, nhóm đồ án đã hoàn thành các mục tiêu đề ra với những kết quả cụ thể như sau:

Một là, chương trình đã triển khai tương đối hoàn chỉnh cấu trúc dữ liệu Cây con trái - Em phải (LCRS) để mô hình hóa các mối quan hệ gia đình đa phân phức tạp một cách tối ưu về bộ nhớ. Hệ thống đã được mô-đun hóa hoàn chỉnh thành các phân hệ độc lập, giúp mã nguồn trở nên sáng sủa, dễ bảo trì và mở rộng.

Hai là, nhóm đã xây dựng và tối ưu hóa thành công thuật toán xác định quan hệ dựa trên Tổ tiên chung gần nhất (LCA) và hệ tọa độ thế hệ (d_A, d_B). Hệ thống có khả năng nhận diện chính xác các vai vế trong tiếng Việt từ cơ bản (Cha, Mẹ, Con) đến phức tạp (Anh/Chị/Em họ, Cháu bàng hệ, Hậu duệ xa).

Ba là, chương trình đáp ứng tốt các nhu cầu thực tế như lưu trữ dữ liệu lâu dài qua tệp nhị phân, nhập liệu hàng loạt từ tệp văn bản và cung cấp giao diện tương tác trực quan qua dòng lệnh.

Bốn là, thông qua đồ án này, nhóm đã làm chủ được kỹ năng quản lý bộ nhớ động với con trỏ trong C++, hiểu sâu về liên kết mô-đun và cách tổ chức một dự án phần mềm, làm tiền đề và đúc kết những kinh nghiệm quý báu cho những đồ án chuyên sâu và phức tạp tiếp theo.

5.2. Hướng phát triển

Dù đã đạt được những kết quả khả quan, hệ thống vẫn còn nhiều tiềm năng để nâng cấp và hoàn thiện trong tương lai:

1. Thay thế giao diện dòng lệnh (CLI) bằng các framework đồ họa như Qt hoặc SFML. Việc này sẽ cho phép hiển thị sơ đồ cây gia phả dưới dạng hình ảnh trực quan, có thể kéo thả và thu phóng, giúp người dùng phổ thông dễ dàng tiếp cận hơn.
2. Thay vì lưu trữ tệp tin cục bộ, hệ thống có thể chuyển sang sử dụng SQLite hoặc PostgreSQL. Điều này giúp quản lý hàng nghìn thành viên một cách nhanh chóng, hỗ trợ các truy vấn phức tạp và đảm bảo an toàn dữ liệu cao hơn.
3. Phát triển tính năng đính kèm hình ảnh, video ký ức hoặc các tài liệu quét (scan)

sắc phong, gia phả cổ cho từng cá nhân, biến phần mềm thành một “Bảo tàng số” của dòng họ.

4. Triển khai thuật toán khớp chuỗi gần đúng (fuzzy search) để hỗ trợ tìm kiếm khi người dùng nhập tên không chính xác hoặc không đủ dấu, đồng thời bổ sung bộ lọc theo ngày sinh, nghề nghiệp hoặc địa điểm.

5. Phát triển phiên bản Web hoặc Mobile, hỗ trợ đồng bộ hóa dữ liệu qua Cloud để các thành viên trong dòng họ ở khắp nơi đều có thể cùng đóng góp và tra cứu thông tin chung.

TÀI LIỆU THAM KHẢO

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
- [2] Đặng Thiên Bình (n.d.). *Bài giảng Cấu trúc dữ liệu*. Lưu hành nội bộ.
- [3] chahelgupta. (n.d.). *DSA-Project-FamilyTree: C++ program for building and visualizing a hierarchical family tree*. GitHub. <https://github.com/chahelgupta/DSA-Project-FamilyTree>
- [4] praveenmylavarapu. (n.d.). *family-tree: A family tree program in C++ using n-ary tree*. GitHub. <https://github.com/praveenmylavarapu/family-tree>
- [5] tenouk. (n.d.). *Constructing a very simple family structure tree using the C++ inheritance principles*. tenouk.com. <https://www.tenouk.com/cpluspluscodesnippet/cplusplusclassinheritancefamilytree.html>
- [6] Trường Đại học Quy Nhơn. (2025). *Mẫu code C++ cho cây nhị phân và cây gia phả*. Studocu. <https://www.studocu.vn/vn/document/dai-hoc-quy-nhon/li-luan-giang-day/mau-code-c-cho-cay-nhi-phan-va-cay-gia-pha-kcj/148579038>
- [7] homielab. (n.d.). *Gia Phả OS (Gia Phả Open Source)*. Github. <https://github.com/homielab/giapha-os>
- [8] Wikipedia contributors. (2025). *Kinship*. Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/wiki/Kinship>

PHỤ LỤC

A. Cài đặt các chức năng lõi của chương trình

A.1. Cấu trúc dữ liệu và các header

Listing 1: Tập tin DataStructure.h

```
1 #ifndef DATA_STRUCTURE_H
2 #define DATA_STRUCTURE_H
3
4 #include <iostream>
5 #include <string>
6 #include <vector>
7
8 #define RESET   "\033[0m"
9 #define BOLD    "\033[1m"
10 #define RED     "\033[31m"
11 #define GREEN   "\033[32m"
12 #define DUT_BLUE "\033[1;34m"
13 #define ORANGE  "\033[38;5;208m"
14 #define CYAN    "\033[36m"
15 #define HIGHLIGHT "\033[1;37;44m"
16 #define YELLOW  "\033[38;5;226m"
17
18 using namespace std;
19
20 struct Person {
21     int id;
22     string name;
23     string gender;
24     string birthday ;
25     string job;
26     string deathDay;
27     string spouseName;
28     int numChildren;
29     int birthYear ;
30
31     Person* parent = nullptr ;
```

```

32     Person* firstChild = nullptr ;
33     Person* nextSibling = nullptr ;
34
35     Person( string _name, string _gender, int _birthYear , string _birthday , Person*
        _parent = nullptr );
36
37     static int extractYear ( string bday) {
38         if (bday.length () < 4) return 0;
39         try {
40             return stoi (bday.substr (bday.length () - 4));
41         } catch (...) { return 0; }
42     }
43 };
44
45 struct SearchResult {
46     Person* node;
47     bool isSpouse;
48 };
49
50 struct PersonRecord {
51     int id;
52     int parentId ;
53     char name[50], gender[10], birthday [20], job [50], deathDay[20], spouseName[50];
54     int numChildren, birthYear ;
55 };
56
57 extern Person* root;
58 extern int global_id_counter ;
59
60 #endif

```

Listing 2: Tập tin TreeManager.h

```

1 #ifndef TREE_MANAGER_H
2 #define TREE_MANAGER_H
3
4 #include "DataStructure .h"
5

```

```

6 Person* CreateFamily( string name, string gender, string bday, int birthYear );
7 Person* AddChild(Person* parent, string name, string bday, string gender);
8 void LinkChildSorted(Person* parent, Person* child);
9 void DisplayTree(Person* current, int level);
10 void ShowDetail(SearchResult p);
11 void UpdatePersonInfo(Person* p);
12
13 void FreeTree(Person* current);
14 void ClearCurrentFamily();
15
16 #endif

```

Listing 3: Tập tin SearchEngine.h

```

1 #ifndef SEARCHENGINE_H
2 #define SEARCHENGINE_H
3
4 #include "TreeManager.h"
5 #include <string>
6 #include <vector>
7
8 using namespace std;
9
10 extern Person* root;
11
12 void FindAllMatches(Person* current, string targetName, vector<SearchResult>&
    results);
13 SearchResult SelectPersonWithProxy( string targetName);
14 Person* FindFirstMatchForImport(Person* rootNode, string targetName);
15
16 #endif

```

Listing 4: Tập tin Relationship.h

```

1 #ifndef RELATIONSHIP_H
2 #define RELATIONSHIP_H
3
4 #include "DataStructure.h"
5

```

```

6 vector<Person*> GetPathToRoot(Person* p);
7 string DetermineRelationship(Person* pA, bool isSpouseA, Person* pB, bool isSpouseB)
8     ;
9 Person* GetAncestor(Person* start , int dA);
10 void CollectAtDepth(Person* root, int targetDepth, int currentDepth, vector<Person
11     *>& results);
12 vector<Person*> FindRelativesByRelationship(Person* startPerson , string relationName
13     );
14 vector<string> FindRelativesNames(Person* startPerson , string relationName);
15
16 #endif

```

Listing 5: Tập tin FileHandler.h

```

1 #ifndef FILE_HANDLER_H
2 #define FILE_HANDLER_H
3
4 #include "DataStructure.h"
5 #include "TreeManager.h"
6 #include "SearchEngine.h"
7 #include "Relationship.h"
8
9 string trim(const string & str);
10 void FlattenTree(Person* current, vector<PersonRecord>& listRecords);
11 void SaveTreeToFile(string filename);
12 void ImportFromTextFile(string filename);
13 void LoadTreeFromFile(string filename);
14
15 #endif

```

A.2. Các chương trình thi hành các chức năng cốt lõi

Listing 6: Tập tin TreeManager.cpp

```

1 #include "TreeManager.h"
2 #include <iostream>
3 #include <string>
4 #include <vector>

```



```

5 #include <fstream>
6 #include <cstring>
7 #include <sstream>
8 #include <unordered_map>
9 #include <algorithm>
10 #include <cstdlib>
11
12 using namespace std;
13
14 Person* root = nullptr ;
15 int global_id_counter = 1;
16
17
18 Person::Person( string _name, string _gender, int _birthYear, string _birthday,
19     Person* _parent) {
20     id = global_id_counter++;
21     name = _name;
22     gender = _gender;
23     birthYear = _birthYear;
24     birthday = _birthday;
25     parent = _parent;
26     job = "Unknown";
27     deathDay = "N/A";
28     spouseName = "None";
29     numChildren = 0;
30     firstChild = nullptr;
31     nextSibling = nullptr;
32 }
33
34 Person* CreateFamily( string name, string gender, string bday, int birthYear ) {
35     if ( root != nullptr ) return root;
36     root = new Person(name, gender, birthYear, bday, nullptr);
37     return root;
38 }
39
40 Person* AddChild(Person* parent, string name, string bday, string gender) {

```

```

41     if (parent == nullptr || parent->gender == "Nu") return nullptr ;
42
43     int nSinh = Person::extractYear (bday);
44
45     Person* newMember = new Person(name, gender, nSinh, bday, parent) ;
46
47     if (parent->firstChild == nullptr) {
48         parent->firstChild = newMember;
49     }
50     else if (newMember->birthYear < parent->firstChild->birthYear) {
51         newMember->nextSibling = parent->firstChild;
52         parent->firstChild = newMember;
53     }
54     else {
55         Person* curr = parent->firstChild ;
56         while (curr->nextSibling != nullptr && curr->nextSibling->birthYear <=
newMember->birthYear) {
57             curr = curr->nextSibling;
58         }
59         newMember->nextSibling = curr->nextSibling;
60         curr->nextSibling = newMember;
61     }
62     return newMember;
63 }
64
65 void LinkChildSorted(Person* parent, Person* child) {
66     if (parent == nullptr || child == nullptr) return;
67     child->parent = parent;
68
69     if (parent->firstChild == nullptr) {
70         parent->firstChild = child;
71     }
72     else if (child->birthYear < parent->firstChild->birthYear) {
73         child->nextSibling = parent->firstChild;
74         parent->firstChild = child;
75     }
76     else {

```

```

77     Person* curr = parent->firstChild ;
78     while ( curr->nextSibling != nullptr && curr->nextSibling->birthYear <= child
->birthYear) {
79         curr = curr->nextSibling;
80     }
81     child->nextSibling = curr->nextSibling;
82     curr->nextSibling = child ;
83 }
84 }
85
86 void DisplayTree(Person* current , int level) {
87     if ( current == nullptr ) return;
88     for (int i = 0; i < level ; i++) cout << "    ";
89     cout << "|--- [" << current->id << "]" << current->name << "(" << current->
gender << ")" << endl;
90     DisplayTree( current->firstChild , level + 1);
91     DisplayTree( current->nextSibling , level );
92 }
93
94 void ShowDetail(SearchResult res) {
95     if (res.node == nullptr) return;
96
97     Person* p = res.node;
98     cout << "\n" << DUT_BLUE << "===== THÔNG TIN CHI TIẾT ====="
<< RESET << endl;
99
100    if (res.isSpouse) {
101        cout << BOLD << "Ho và ten: " << RESET << GREEN << p->spouseName <<
RESET << " (Phu nhân)" << endl;
102        cout << BOLD << "Giới tính: " << RESET << "Nu" << endl;
103        cout << BOLD << "Phụ quan: " << RESET << p->name << " (ID: " << p->id << "
)" << endl;
104        cout << BOLD << "Gia đình: " << RESET << "Nhánh của " << (p->parent ? p->
parent->name : "Ông Tổ") << endl;
105    }
106    else {

```

```

107     cout << BOLD << "Ho va ten: " << RESET << GREEN << p->name << RESET
    << endl;
108     cout << BOLD << "ID:      " << RESET << p->id << endl;
109     cout << BOLD << "Gioi tinh: " << RESET << p->gender << endl;
110     cout << BOLD << "Ngày sinh: " << RESET << p->birthday << endl;
111     if (p->gender == "Nu") {
112         cout << BOLD << "Phu quan: " << RESET << p->spouseName << endl;
113     }
114     else if (!p->spouseName.empty()) {
115         cout << BOLD << "Phu nhan: " << RESET << p->spouseName << endl;
116     }
117
118     if (p->parent != nullptr) {
119         cout << BOLD << "Than sinh: " << RESET << p->parent->name << endl;
120     }
121 }
122 cout << DUT_BLUE << "===== " <<
    RESET << endl;
123 }
124
125 void UpdatePersonInfo(Person* p) {
126     if (p == nullptr) return;
127
128     string temp;
129
130     cout << "\n--- Cap nhat thong tin cho: [" << p->id << "] " << p->name << " ---\n";
131     cout << "(Nhan Enter neu muon giu nguyen thong tin cu)\n";
132
133     cout << "Ngày sinh hien tai [" << p->birthday << "]: ";
134     getline (cin, temp);
135     if (!temp.empty()) {
136         p->birthday = temp;
137         p->birthYear = Person::extractYear (temp);
138     }
139
140     cout << "Nghề nghiệp hien tai [" << p->job << "]: ";

```

```

141     getline ( cin , temp);
142     if (!temp.empty()) p->job = temp;
143
144     cout << "Ngày mất hiện tại [" << p->deathDay << "]: ";
145     getline ( cin , temp);
146     if (!temp.empty()) p->deathDay = temp;
147
148     cout << "Tên vợ/chồng hiện tại [" << p->spouseName << "]: ";
149     getline ( cin , temp);
150     if (!temp.empty()) p->spouseName = temp;
151
152     if (p->gender == "Nu") {
153         cout << "Số con hiện tại [" << p->numChildren << "]: ";
154         getline ( cin , temp);
155         if (!temp.empty()) {
156             p->numChildren = stoi(temp);
157         }
158     }
159
160     cout << "\n[OK] Cập nhật thành công!\n";
161 }
162
163 void FreeTree(Person* current) {
164     if (current == nullptr) return;
165     FreeTree( current->firstChild );
166     FreeTree( current->nextSibling);
167     delete current ;
168 }
169
170 void ClearCurrentFamily() {
171     if (root == nullptr) {
172         cout << "[Thông báo] Gia phả hiện đang trống , không cần xóa.\n";
173         return;
174     }
175
176     FreeTree(root);
177

```

```

178     root = nullptr ;
179     global_id_counter = 1;
180
181     cout << "[OK] Da xoa sach toan bo du lieu gia pha tren RAM!\n";
182     cout << "ID he thong da duoc reset ve 1.\n";
183 }

```

Listing 7: Tập tin SearchEngine.cpp

```

1  #include "SearchEngine.h"
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  #include <algorithm>
6
7  using namespace std;
8
9
10 void FindAllMatches(Person* current, string targetName, vector<SearchResult>&
    results ) {
11     if ( current == nullptr ) return;
12
13     if ( current->name == targetName) {
14         results .push_back({ current, false });
15     }
16
17     if ( current->spouseName == targetName) {
18         results .push_back({ current, true });
19     }
20
21     FindAllMatches(current->firstChild , targetName, results );
22     FindAllMatches(current->nextSibling, targetName, results );
23 }
24
25 SearchResult SelectPersonWithProxy( string targetName) {
26     vector<SearchResult> results ;
27     FindAllMatches(root, targetName, results );
28

```

```

29     if ( results .empty() ) {
30         cout << "\n\033[1;31m[Loi] Khong tim thay ai co ten la '" << targetName << "
        '\033[0m\n";
31         return { nullptr , false };
32     }
33
34     if ( results .size () == 1) return results [0];
35
36     cout << "\n\033[1;36m[Chu y] Tim thay '" << results .size () << " ket qua cho ten '"
        << targetName << " '\033[0m\n";
37     cout << "-----\n";
38     for ( size_t i = 0; i < results .size () ; i++) {
39         cout << i + 1 << ". ";
40         if ( results [i].isSpouse) {
41             cout << "[VO] '" << targetName << " (Vo cua '" << results [i].node->name
            << ")";
42         } else {
43             cout << "[NUT] '" << results [i].node->name << " (ID: '" << results [i].
            node->id << ")";
44         }
45
46         if ( results [i].node->parent != nullptr )
47             cout << " | Nhanh: '" << results [i].node->parent->name;
48         cout << endl;
49     }
50     cout << "-----\n";
51
52     int choice = 0;
53     while (true) {
54         cout << "Nhap so thu tu de chon: ";
55         if (!( cin >> choice)) {
56             cin . clear () ;
57             cin . ignore(1000, '\n');
58             continue;
59         }
60         if (choice >= 1 && choice <= (int) results .size () ) {
61             cin . ignore(1000, '\n');

```

```

62         return results [choice - 1];
63     }
64     cout << "Lua chon khong hop le !\n";
65 }
66 }
67
68 Person* FindFirstMatchForImport(Person* rootNode, string targetName) {
69     vector<SearchResult> results ;
70     FindAllMatches(rootNode, targetName, results );
71
72     if ( results .empty()) return nullptr ;
73
74     for(auto res : results ) {
75         if (! res .isSpouse) return res .node;
76     }
77     return results [0].node;
78 }

```

Listing 8: Tập tin Relationship.cpp

```

1  #include "Relationship .h"
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  #include <fstream>
6  #include <cstring>
7  #include <sstream>
8  #include <unordered_map>
9  #include <algorithm>
10 #include <cstdlib>
11
12 using namespace std;
13
14 vector<Person*> GetPathToRoot(Person* p) {
15     vector<Person*> path;
16     while (p != nullptr) {
17         path.push_back(p);
18         p = p->parent;

```



```

19     }
20     reverse (path.begin() , path.end());
21     return path;
22 }
23
24 string DetermineRelationship(Person* pA, bool isSpouseA, Person* pB, bool isSpouseB)
25 {
26     if (pA == nullptr || pB == nullptr) return "Loi: Nguoi khong ton tai!";
27
28     if (pA == pB && isSpouseA == isSpouseB) return "Chinh la mot nguoi!";
29     if (pA == pB && isSpouseA != isSpouseB) return "Quan he: Vo chong";
30
31     vector<Person*> pathA = GetPathToRoot(pA);
32     vector<Person*> pathB = GetPathToRoot(pB);
33
34     int lcaIndex = -1;
35     int minLen = min((int)pathA.size() , (int)pathB.size());
36     for (int i = 0; i < minLen; i++) {
37         if (pathA[i] == pathB[i]) lcaIndex = i;
38         else break;
39     }
40
41     if (lcaIndex == -1) return "Khong co quan he ho hang";
42
43     int dA = (pathA.size() - 1) - lcaIndex;
44     int dB = (pathB.size() - 1) - lcaIndex;
45     Person* lca = pathA[lcaIndex];
46
47     if (dA == 0) {
48         if (dB == 1) return isSpouseA ? "Me" : "Cha";
49         if (dB == 2) return isSpouseA ? "Ba noi" : "Ong noi";
50         if (dB == 3) return isSpouseA ? "Ba co" : "Ong co";
51         return isSpouseA ? "To tien (Nu)" : "To tien (Nam)";
52     }
53
54     if (dB == 0) {
55         if (isSpouseA) return "Nang dau";

```

```

55     if (dA == 1) return "Con";
56     if (dA == 2) return "Chau";
57     if (dA == 3) return "Chat";
58     return "Hau due";
59 }
60
61 if (dA == 1 && dB == 1) {
62     bool aIsOlder = false;
63     Person* curr = lca->firstChild;
64     while (curr != nullptr) {
65         if (curr == pA) { aIsOlder = true; break; }
66         if (curr == pB) { aIsOlder = false; break; }
67         curr = curr->nextSibling;
68     }
69
70     if (isSpouseA) {
71         if (aIsOlder) return (pA->gender == "Nam") ? "Chi dau" : "Anh re";
72         else return (pA->gender == "Nam") ? "Em dau" : "Em re";
73     }
74
75     if (aIsOlder) return (pA->gender == "Nam") ? "Anh trai" : "Chi gai";
76     return (pA->gender == "Nam") ? "Em trai" : "Em gai";
77 }
78
79 if (dA == 1 && dB == 2) {
80     Person* aNode = pathA[lcaIndex + 1];
81     Person* parentOfB = pathB[lcaIndex + 1];
82
83     bool aIsOlder = false;
84     Person* curr = lca->firstChild;
85     while (curr != nullptr) {
86         if (curr == aNode) { aIsOlder = true; break; }
87         if (curr == parentOfB) { aIsOlder = false; break; }
88         curr = curr->nextSibling;
89     }
90
91     if (isSpouseA) {

```

```

92         if (aIsOlder) return "Bac gai (Vo bac)";
93         return (pA->gender == "Nam") ? "Thim (Vo chu)" : "Duong (Chong co)";
94     }
95
96     if (aIsOlder) return (pA->gender == "Nam") ? "Bac trai" : "Bac gai";
97     return (pA->gender == "Nam") ? "Chu" : "Co";
98 }
99
100 return "Ho hang xa (Cach " + to_string (dA + dB) + " doi)";
101 }
102
103 // 4B. TIM NGUOI THEO QUAN HE
104
105 Person* GetAncestor(Person* start , int dA) {
106     Person* curr = start ;
107     for (int i = 0; i < dA; i++) {
108         if (curr != nullptr) curr = curr->parent;
109     }
110     return curr;
111 }
112
113 void CollectAtDepth(Person* root, int targetDepth, int currentDepth, vector<Person
114     *>& results) {
115     if (root == nullptr) return;
116
117     if (currentDepth == targetDepth) {
118         results.push_back(root);
119         return;
120     }
121
122     Person* child = root->firstChild;
123     while (child != nullptr) {
124         CollectAtDepth(child, targetDepth, currentDepth + 1, results);
125         child = child->nextSibling;
126     }
127 }

```

```

128 vector<Person*> FindRelativesByRelationship(Person* startPerson , string relationName
    ) {
129     vector<Person*> finalResults ;
130     if ( startPerson == nullptr ) return finalResults ;
131
132     int dA = -1, dB = -1;
133
134     if (relationName == "Cha" || relationName == "Me") { dA = 1; dB = 0; }
135     else if (relationName == "Ong" || relationName == "Ba") { dA = 2; dB = 0; }
136     else if (relationName == "Anh trai" || relationName == "Chi gái" ||
137             relationName == "Em trai" || relationName == "Em gái") { dA = 1; dB =
138             1; }
139     else if (relationName == "Bac trai" || relationName == "Bac gái" ||
140             relationName == "Chu" || relationName == "Co" || relationName == "Di")
141     { dA = 2; dB = 1; }
142     else if (relationName == "Anh/Chi/Em ho") { dA = 2; dB = 2; }
143     else if (relationName == "Con") { dA = 0; dB = 1; }
144     else if (relationName == "Chau") { dA = 0; dB = 2; }
145     else if (relationName == "Hau due cua Anh/Chi/Em") { dA = 1; dB = -2;}
146     else {
147         cout << "[Loi] Chua ho tro truy van danh xung: " << relationName << "\n";
148         return finalResults ;
149     }
150
151     Person* ancestor = GetAncestor( startPerson , dA);
152     if ( ancestor == nullptr ) return finalResults ;
153
154     vector<Person*> candidates;
155     if (dB == -2) {
156         Person* siblingOfStart = ancestor->firstChild ;
157         while ( siblingOfStart != nullptr ) {
158             if ( siblingOfStart != startPerson ) {
159                 for (int i = 2; i <= 5; i++) {
160                     CollectAtDepth( siblingOfStart , i, 1, candidates );
161                 }
162             }
163             siblingOfStart = siblingOfStart ->nextSibling;

```

```

162     }
163 } else if (dB == 0) { candidates.push_back(ancestor); }
164 else {
165     Person* child = ancestor->firstChild;
166     while (child != nullptr) {
167         CollectAtDepth(child, dB, 1, candidates);
168         child = child->nextSibling;
169     }
170 }
171
172 for (Person* p : candidates) {
173     if (p == startPerson) continue;
174     bool match = false;
175
176     if (relationName == "Cha" || relationName == "Ong") { if (p->gender == "
Nam") match = true; }
177     else if (relationName == "Me" || relationName == "Ba") { if (p->gender == "
Nu") match = true; }
178     else if (relationName == "Con" || relationName == "Chau" || relationName ==
"Anh/Chi/Em ho") { match = true; }
179
180     else if (dA == 1 && dB == 1) {
181         bool isOlder = false;
182         Person* curr = ancestor->firstChild;
183         while (curr != nullptr) {
184             if (curr == p) { isOlder = true; break; }
185             if (curr == startPerson) { isOlder = false; break; }
186             curr = curr->nextSibling;
187         }
188         if (relationName == "Anh trai" && isOlder && p->gender == "Nam")
match = true;
189         if (relationName == "Chi gái" && isOlder && p->gender == "Nu") match =
true;
190         if (relationName == "Em trai" && !isOlder && p->gender == "Nam")
match = true;
191         if (relationName == "Em gái" && !isOlder && p->gender == "Nu") match =
true;

```

```

192     }
193
194     else if (dA == 2 && dB == 1) {
195         Person* parentOfStart = startPerson->parent;
196         if (p == parentOfStart) continue;
197
198         bool isOlderThanParent = false ;
199         Person* curr = ancestor->firstChild ;
200         while (curr != nullptr) {
201             if (curr == p) { isOlderThanParent = true; break; }
202             if (curr == parentOfStart) { isOlderThanParent = false; break; }
203             curr = curr->nextSibling;
204         }
205         if (relationName == "Bac trai" && isOlderThanParent && p->gender == "
Nam") match = true;
206         if (relationName == "Bac gai" && isOlderThanParent && p->gender == "
Nu") match = true;
207         if (relationName == "Chu" && !isOlderThanParent && p->gender == "Nam"
) match = true;
208         if (relationName == "Co" && !isOlderThanParent && p->gender == "Nu")
match = true;
209         if (relationName == "Di" && !isOlderThanParent && p->gender == "Nu")
match = true;
210     }
211
212     else if (relationName == "Anh/Chi/Em ho" && dA == 2 && dB == 2) {
213         Person* parentOfStart = startPerson->parent;
214
215         if (p->parent == parentOfStart) continue;
216
217         match = true;
218     }
219     else if (relationName == "Hau due cua Anh/Chi/Em") { match = true;}
220
221     if (match) finalResults .push_back(p);
222 }
223

```

```

224     return finalResults ;
225 }
226
227 vector<string> FindRelativesNames(Person* startPerson , string relationName) {
228     vector<string> names;
229     if ( startPerson == nullptr ) return names;
230
231     if (relationName == "Me") {
232         if ( startPerson ->parent != nullptr ) {
233             string sName = startPerson ->parent ->spouseName;
234             if (!sName.empty() && sName != "None" && sName != "N/A" && sName
235 != "Unknown") {
236                 names.push_back(sName + " (Vo của " + startPerson ->parent ->name + "
237 )");
238             }
239         }
240         return names;
241     }
242
243     if (relationName == "Ba") {
244         Person* dad = startPerson ->parent;
245         if (dad != nullptr && dad ->parent != nullptr) {
246             string sName = dad ->parent ->spouseName;
247             if (!sName.empty() && sName != "None" && sName != "N/A" && sName
248 != "Unknown") {
249                 names.push_back(sName + " (Ba noi - Vo của " + dad ->parent ->name +
250 ")");
251             }
252         }
253         return names;
254     }
255
256     vector<Person*> results = FindRelativesByRelationship ( startPerson , relationName)
257 ;
258     for (Person* p : results ) {
259         names.push_back(p ->name);
260     }

```

```

256
257     return names;
258 }

```

Listing 9: Tập tin FileHandler.cpp

```

1  #include "FileHandler.h"
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  #include <fstream>
6  #include <cstring>
7  #include <sstream>
8  #include <unordered_map>
9  #include <algorithm>
10 #include <cstdlib>
11
12 using namespace std;
13
14 string trim(const string & str) {
15     size_t first = str.find_first_not_of(' ');
16     if (string::npos == first) return str;
17     size_t last = str.find_last_not_of(' ');
18     return str.substr(first, (last - first + 1));
19 }
20
21 int safeStoi(string s) {
22     try {
23         return stoi(trim(s));
24     } catch (...) {
25         return 0;
26     }
27 }
28
29 void FlattenTree(Person* current, vector<PersonRecord>& listRecords) {
30     if (current == nullptr) return;
31     PersonRecord rec;
32     rec.id = current->id;

```



```

33     rec.parentId = ( current->parent != nullptr ) ? current->parent->id : 0;
34     rec.birthYear = current->birthYear;
35
36     strncpy( rec.name, current->name.c_str(), 49); rec.name[49] = '\0';
37     strncpy( rec.gender, current->gender.c_str(), 9); rec.gender[9] = '\0';
38     strncpy( rec.birthday, current->birthday.c_str(), 19); rec.birthday[19] = '\0';
39
40     listRecords.push_back(rec);
41     FlattenTree( current->firstChild, listRecords );
42     FlattenTree( current->nextSibling, listRecords );
43 }
44
45 void SaveTreeToFile( string filename ) {
46     if ( root == nullptr ) return;
47     vector<PersonRecord> listRecords;
48     FlattenTree( root, listRecords );
49
50     ofstream outFile( filename, ios::binary);
51     if ( outFile.is_open() ) {
52         int total = listRecords.size();
53         outFile.write((char*)&total, sizeof(int));
54         outFile.write((char*)listRecords.data(), total * sizeof(PersonRecord));
55         outFile.close();
56         cout << "\n[OK] Da luu " << total << " nguoi vao " << filename << " (bao
gom nam sinh)\n";
57     }
58 }
59
60 void ImportFromTextFile( string filename ) {
61     ifstream inFile( filename);
62     if ( !inFile.is_open() ) {
63         cout << "[Loi] Khong the mo file " << filename << ". Haykiem tra lai ten
file!\n";
64         return;
65     }
66
67     if ( root != nullptr ) {

```

```

68     char confirm;
69     cout << "[Canh bao] Gia pha hien tai se bi xoa sach de nap du lieu moi.\n";
70     cout << "Ban co chac chan muon tiep tục? (y/n): ";
71     cin >> confirm;
72     if (confirm != 'y' && confirm != 'Y') {
73         cout << "Huy bo thao tac nap file .\n";
74         return;
75     }
76     ClearCurrentFamily();
77 }
78
79 string line;
80 int count = 0;
81 while ( getline ( inFile , line )) {
82     if ( line.empty()) continue;
83
84     stringstream ss( line );
85     string tenCha, tenCon, gioiTinh, ngaySinh;
86     int namSinh;
87
88     getline (ss, tenCha, ',');
89     getline (ss, tenCon, ',');
90     getline (ss, gioiTinh, ',');
91     getline (ss, ngaySinh, ',');
92
93     tenCha = trim(tenCha);
94     tenCon = trim(tenCon);
95     gioiTinh = trim(gioiTinh);
96     ngaySinh = trim(ngaySinh);
97     namSinh = Person::extractYear (ngaySinh);
98
99     if (tenCha == "None" || tenCha == "") {
100         if (root == nullptr) {
101             CreateFamily(tenCon, gioiTinh, ngaySinh, namSinh);
102             count++;
103         } else {
104             cout << "- Bo qua Ong To [" << tenCon << "] vi gia pha da co goc!\n"

```

```

;
105     }
106     } else {
107         Person* cha = FindFirstMatchForImport(root, tenCha);
108         if (cha != nullptr) {
109             if (cha->gender == "Nam") {
110                 AddChild(cha, tenCon, ngaySinh, gioiTinh);
111                 count++;
112             } else {
113                 cout << "- [Loi] Không thể thêm con cho Nu: '" << tenCha
114                     << "'. Thao tác bị từ chối theo quy tắc gia phả.\n";
115             }
116         } else {
117             cout << "- [Cảnh báo] Không tìm thấy cha: '" << tenCha << "' để
118             thêm con '" << tenCon << "'\n";
119         }
120     }
121
122     inFile.close();
123     cout << "\n[OK] Đã đọc và sắp xếp '" << count << "' thành viên từ file text theo
124     thu từ nam sinh!\n";
125 }
126 void LoadTreeFromFile(string filename) {
127     ifstream inFile (filename, ios::binary);
128     if (!inFile.is_open()) return;
129
130     int total;
131     inFile.read((char*)&total, sizeof(int));
132
133     vector<PersonRecord> listRecords (total);
134     inFile.read((char*)listRecords.data(), total * sizeof(PersonRecord));
135     inFile.close();
136
137     ClearCurrentFamily();
138     unordered_map<int, Person*> idMap;

```

```

139     int maxId = 0;
140
141     for (const auto& rec : listRecords ) {
142         Person* p = new Person(rec.name, rec.gender, rec.birthYear, rec.birthday);
143         p->id = rec.id;
144         p->job = rec.job;
145         p->deathDay = rec.deathDay;
146         p->spouseName = rec.spouseName;
147         p->numChildren = rec.numChildren;
148
149         idMap[p->id] = p;
150         if (p->id > maxId) maxId = p->id;
151     }
152
153     for (const auto& rec : listRecords ) {
154         Person* child = idMap[rec.id];
155         if (rec.parentId == 0) {
156             root = child;
157         } else {
158             if (idMap.count(rec.parentId)) {
159                 Person* pParent = idMap[rec.parentId];
160                 LinkChildSorted(pParent, child);
161             }
162         }
163     }
164
165     global_id_counter = maxId + 1;
166     cout << "\n[OK] Da phuc hoi hoan toan " << total << " thanh vien!\n";
167 }

```

Listing 10: Tập tin main.cpp

```

1 #include "TreeManager.h"
2 #include "SearchEngine.h"
3 #include "FileHandler.h"
4 #include "Relationship.h"
5 #include <iostream>
6 #include <limits>

```

```
7 #include <conio.h>
8 #include <fstream>
9 #include <windows.h>
10
11 using namespace std;
12
13 void clearScreen () {
14 #ifdef _WIN32
15     system("cls");
16 #else
17     system("clear");
18 #endif
19 }
20
21 struct FamilyStats {
22     int totalMembers = 0;
23     int totalSpouses = 0;
24     int maxGen = 0;
25     int maleCount = 0;
26     int femaleCount = 0;
27     int deceasedCount = 0;
28     int genDistribution [20] = {0};
29 };
30
31 void CalculateStats (Person* p, int level , FamilyStats &s) {
32     if (p == nullptr ) return;
33
34     s.totalMembers++;
35     if (p->gender == "Nam") s.maleCount++;
36     else if (p->gender == "Nu") s.femaleCount++;
37
38     if (!p->spouseName.empty() && p->spouseName != "None" && p->spouseName !=
"N/A") {
39         s. totalSpouses ++;
40     }
41
42     if (p->deathDay != "N/A" && p->deathDay != "Unknown" && !p->deathDay.empty
```

```

0) {
43     s.deceasedCount++;
44 }
45
46 if ( level < 20) s. genDistribution [ level ]++;
47 if ( level > s.maxGen) s.maxGen = level;
48
49 CalculateStats (p->firstChild , level + 1, s);
50 CalculateStats (p->nextSibling, level , s);
51 }
52
53 void printHeader() {
54     system("cls");
55
56     const int width = 78;
57
58     cout << DUT_BLUE << BOLD;
59
60     cout << (char)201;
61     for(int i = 0; i < width; i++) cout << (char)205;
62     cout << (char)187 << endl;
63
64     cout << (char)186 << R"(      _____      _      _
        _      )" << " " << (char)186 << endl;
65     cout << (char)186 << R"(    / __ \      | |      ( )
        | |      )" << " " << (char)186 << endl;
66     cout << (char)186 << R"(   | | | | | _ _ _ _ _ | | _ _ _ _ _
        _ _ | | _ _ _ _ )" << " " << (char)186 << endl;
67     cout << (char)186 << R"(   | | | | | | / _ ' | ' _ \ | | | | | / _ ' | / _ ' |
        | ' _ \ | ' _ \ / _ ' | )" << " " << (char)186 << endl;
68     cout << (char)186 << R"(   | | _ | | _ | ( | | | | | | _ | | ( | | | ( | | |
        | | ) | | | | ( | | )" << " " << (char)186 << endl;
69     cout << (char)186 << R"(   \ _ _ \ \ _ , \ _ , | _ | _ \ _ , | \ _ , | _ \ _ , |
        | . _ / | _ | _ \ _ , | )" << " " << (char)186 << endl;
70     cout << (char)186 << R"(      _ / | _ / |
        | |      )" << " " << (char)186 << endl;
71     cout << (char)186 << R"(      | / | /

```

```

72 |         |_) << " " << (char)186 << endl;
73 |
74 |     cout << (char)199;
75 |     for(int i = 0; i < width; i++) cout << (char)196;
76 |     cout << (char)182 << endl;
77 |
78 |     string subTitle = "PBL 1 – Do an lap trinh tinh toan";
79 |     string subTitle2 = "SVTH: Nguyen Dinh Hai Dang va Le Ba Son – Lop 25T_KHDL";
80 |     int padding = (width – subTitle.length()) / 2;
81 |     int padding2 = (width – subTitle2.length()) / 2;
82 |
83 |     cout << (char)186;
84 |     for(int i = 0; i < padding; i++) cout << " ";
85 |     cout << YELLOW << subTitle << DUT_BLUE;
86 |     for(int i = 0; i < width – subTitle.length() – padding; i++) cout << " ";
87 |     cout << (char)186 << endl;
88 |
89 |     cout << (char)186;
90 |     for (int i = 0; i < padding2; i++) cout << " ";
91 |     cout << YELLOW << subTitle2 << DUT_BLUE;
92 |     for (int i = 0; i < width – subTitle2.length() – padding2; i++) cout << " ";
93 |     cout << (char)186 << endl;
94 |
95 |     cout << (char)200;
96 |     for(int i = 0; i < width; i++) cout << (char)205;
97 |     cout << (char)188 << endl;
98 |
99 |     cout << RESET;
100 | }
101 | void showInteractiveMenu(int selectedIndex) {
102 |     system("cls");
103 |     printHeader();
104 |
105 |     vector<string> options = {
106 |         "0. Thoat chuong trinh",
107 |         "1. Hien thi cay gia pha",

```

```

108     "2. Tim kiem & Xem chi tiet",
109     "3. Kiem tra quan he ho hang",
110     "4. Tim nguoi theo quan he",
111     "5. Luu du lieu xuong file",
112     "6. Cap nhat thong tin chi tiet",
113     "7. Nhap lieu hang loat tu file",
114     "8. Thong ke"
115 };
116
117 for (int i = 0; i < options.size(); ++i) {
118     if (i == selectedIndex) {
119
120         cout << HIGHLIGHT << " >" << options[i] << " " << RESET << endl;
121     } else {
122         cout << " " << options[i] << endl;
123     }
124 }
125 cout << "\n" << DUT_BLUE << "(Dung mui ten Len/Xuong va Enter de chon)" <<
RESET;
126 }
127
128 int getMenuChoice() {
129     int selectedIndex = 1;
130     int numOptions = 8;
131
132     while (true) {
133         showInteractiveMenu(selectedIndex);
134
135         int key = _getch();
136         if (key == 224) {
137             key = _getch();
138             if (key == 72) {
139                 selectedIndex--;
140                 if (selectedIndex < 0) selectedIndex = numOptions;
141             }
142             else if (key == 80) {
143                 selectedIndex++;

```



```

144         if ( selectedIndex > numOptions) selectedIndex = 0;
145     }
146 }
147 else if (key == 13) {
148     Beep(3000, 400);
149     return selectedIndex ;
150 }
151 else if (key == '0') {
152     Beep(3000, 400);
153     return 0;
154 }
155 }
156 }
157
158 void pressAnyKey() {
159     cout << "\n-----";
160     cout << "\nAn [Enter] de quay lai menu chính ... ";
161     cin.ignore( numeric_limits<streamsize>::max(), '\n');
162     cin.get();
163 }
164
165 bool askToContinue( string functionName) {
166     cout << "\n" << DUT_BLUE << "
167     -----" << RESET;
168     cout << "\n" << ORANGE << "Ban co muon tiep tục " << BOLD << functionName <<
169     RESET << ORANGE << " khong?" << RESET;
170     cout << "\n" << GREEN << "[Y]: Tiep tục" << RESET << " | " << RED << "[N]: Ve
171     Menu" << RESET;
172
173     while (true) {
174         char ch = _getch();
175         if (ch == 'y' || ch == 'Y') {
176             Beep(1000, 100);
177             return true;
178         }
179         if (ch == 'n' || ch == 'N' || ch == 27) {
180             Beep(750, 150);

```

```

178         return false ;
179     }
180 }
181 }
182
183 void InitFixedData () {
184     cout << "[He thong] Dang khoi tao bo du lieu gia pha mau ...\n";
185     sleep (2);
186     Person* to = CreateFamily("Nguyen Van To", "Nam", "19/09/1928", 1928);
187     Person* hung = AddChild(to, "Nguyen Van Long", "29/03/1949", "Nam");
188     Person* vanh = AddChild(to, "Nguyen Van Vanh", "01/01/1946", "Nam");
189 }
190
191
192 int main() {
193     InitFixedData ();
194
195     string binaryFile = "GiaPha_Test.dat";
196     int luaChon;
197
198     do {
199         clearScreen () ;
200         luaChon = getMenuChoice();
201         switch (luaChon) {
202             case 1:
203                 clearScreen () ;
204                 cout << "\n—— CAY GIA PHA HIEN TAI ——\n";
205                 DisplayTree(root, 0);
206                 pressAnyKey();
207                 break;
208             case 2: {
209                 do {
210                     clearScreen () ;
211                     string ten;
212                     cout << "Nhap ten can xem chi tiet ( viet dung chinh ta ten cua\n";
213                     getline (cin, ten);

```

```

214         SearchResult res = SelectPersonWithProxy(ten);
215         if (res.node != nullptr) {
216             ShowDetail(res);
217         }
218     } while (askToContinue("tim kiem va xem chi tiet thanh vien"));
219     break;
220 }
221 case 3: {
222     do {
223         clearScreen();
224         string ten1, ten2;
225         cout << "Nhap ten nguoi thu nhat ( viet dung chinh ta ten cua
nguoi do):\n"; getline(cin, ten1);
226         SearchResult resA = SelectPersonWithProxy(ten1);
227
228         if (resA.node == nullptr) {
229             cout << "Vui long kiem tra lai ten nguoi thu nhat!" << endl;
230             system("pause");
231             continue;
232         }
233
234         cout << "Nhap ten nguoi thu hai ( viet dung chinh ta ten cua
nguoi do):\n"; getline(cin, ten2);
235         SearchResult resB = SelectPersonWithProxy(ten2);
236
237         if (resB.node == nullptr) {
238             cout << "Vui long kiem tra lai ten nguoi thu hai!" << endl;
239             system("pause");
240             continue;
241         }
242
243         if (resA.node && resB.node) {
244             string quanHe = DetermineRelationship(resA.node, resA.
isSpouse, resB.node, resB.isSpouse);
245
246             if (ten1 == ten2) {
247                 cout << BOLD << quanHe << endl;

```

```

248         }
249         else {
250             cout << "\n" << GREEN << BOLD << "==" << KET QUA: "
<< RESET;
251             cout << BOLD << ten1 << RESET << " la " << RED <<
BOLD << quanHe << RESET;
252             cout << " cua " << BOLD << ten2 << RESET << endl;
253         }
254
255     }
256
257     } while (askToContinue("kiem tra quan he ho hang"));
258     break;
259 }
260
261 case 4: {
262     do {
263         clearScreen ();
264         string tenNguoi;
265         cout << "\n—— TIM NGUOI THEO QUAN HE ——\n";
266         cout << "Nhap ten nguoi lam moc (viet dung chinh ta ten cua
nguoi do):\n";
267         getline (cin , tenNguoi);
268
269         SearchResult res = SelectPersonWithProxy(tenNguoi);
270         if (res.node == nullptr) break;
271
272         Person* moc = res.node;
273
274         cout << "\n===== CHON QUAN HE CAN TIM
=====
\n";
275         cout << "    1. Cha            2. Me            3. Ong            4. Ba\
n";
276         cout << "    5. Anh trai        6. Chi gai        7. Em trai        8. Em
gai\n";
277         cout << "    9. Bac trai        10. Bac gai        11. Chu            12. Co
13. Di\n";

```

```

278         cout << " 14. Anh/Chi/Em ho          15. Con          16.
Chau\n";
279         cout << " 17. Chau/Chat /...  (Hau due cua Anh/Chi/Em)\n";
280         cout << "
===== \n";
281
282         int luaChon;
283         string quanHe = "";
284
285         while (true) {
286             cout << "Nhap lua chon (1-17): ";
287             cin >> luaChon;
288             cin.ignore(256, '\n');
289             switch (luaChon) {
290                 case 1: quanHe = "Cha"; break;
291                 case 2: quanHe = "Me"; break;
292                 case 3: quanHe = "Ong"; break;
293                 case 4: quanHe = "Ba"; break;
294                 case 5: quanHe = "Anh trai"; break;
295                 case 6: quanHe = "Chi gai"; break;
296                 case 7: quanHe = "Em trai"; break;
297                 case 8: quanHe = "Em gai"; break;
298                 case 9: quanHe = "Bac trai"; break;
299                 case 10: quanHe = "Bac gai"; break;
300                 case 11: quanHe = "Chu"; break;
301                 case 12: quanHe = "Co"; break;
302                 case 13: quanHe = "Di"; break;
303                 case 14: quanHe = "Anh/Chi/Em ho"; break;
304                 case 15: quanHe = "Con"; break;
305                 case 16: quanHe = "Chau"; break;
306                 case 17: quanHe = "Hau due cua Anh/Chi/Em"; break;
307                 default: cout << "Lua chon khong hop le , vui long nhap
lai !\n"; continue;
308             }
309             break;
310         }
311

```

```

312         vector<string> ketQua = FindRelativesNames(moc, quanHe);
313
314         if (ketQua.empty()) {
315             cout << "\n" << RED << "=> Không tìm thấy ai là " <<
quanHe << " của " << moc->name << RESET << endl;
316         } else {
317             cout << "\n" << GREEN << "=> Tìm thấy " << ketQua.size()
<< " " << quanHe << " của " << moc->name << ":" << RESET << endl;
318             cout << "
-----\n";
319             for (const string & ten : ketQua) {
320                 cout << " - " << BOLD << ten << RESET << endl;
321             }
322             cout << "
-----\n";
323         }
324     } while (askToContinue("tim nguoi theo quan he"));
325     break;
326 }
327 case 5: {
328     clearScreen ();
329     if (root == nullptr) {
330         cout << RED << "Loi: Gia pha hien tai dang trong , khong co du
lieu de luu!" << RESET << endl;
331         pressAnyKey();
332         break;
333     }
334
335     string userFileName;
336     bool isDuplicate = true;
337
338     while ( isDuplicate ) {
339         cout << YELLOW << "—— LUU DU LIEU GIA PHA ——" <<
RESET << endl;
340         cout << "Nhap ten file muon luu (VD: GiaPhaV2.dat): ";
341         getline (cin , userFileName);
342

```

```

343         if (userFileName.length() < 4 || userFileName.substr (
userFileName.length() - 4) != ".dat") {
344             userFileName += ".dat";
345         }
346
347         ifstream checkFile(userFileName);
348         if (checkFile.is_open()) {
349             cout << RED << "[Canh bao] File '" << userFileName << "' da
ton tai !" << RESET << endl;
350             cout << "Vui long dat mot ten khac de tranh ghi de du lieu
cu.\n" << endl;
351             checkFile.close();
352         } else {
353             isDuplicate = false;
354         }
355     }
356
357     SaveTreeToFile(userFileName);
358     pressAnyKey();
359     break;
360 }
361
362 case 6: {
363     do {
364         clearScreen();
365         string ten;
366         cout << "Nhap ten nguoi can cap nhat ( viet dung chinh ta ten
cua nguoi do):\n";
367         getline (cin, ten);
368
369         SearchResult res = SelectPersonWithProxy(ten);
370
371         if (res.node != nullptr) {
372
373             if (res.isSpouse) {
374                 cout << "Dang cap nhat thong tin cho Phu nhan: " <<
res.node->spouseName << endl;

```

```

375         cout << "Nhap ten moi: ";
376         getline ( cin , res .node->spouseName);
377     }
378     else {
379         UpdatePersonInfo(res .node);
380     }
381
382     cout << "\n—— THÔNG TIN SAU KHI CAP NHAT ——";
383
384     ShowDetail(res);
385 }
386 else {
387     cout << RED << "Khong tim thay nguoi nay trong gia pha!"
<< RESET << endl;
388 }
389 } while (askToContinue("cap nhat thong tin"));
390 break;
391 }
392
393 case 7: {
394     do {
395         clearScreen ();
396         cout << YELLOW << BOLD << "===== NAP DU
LIEU GIA PHA =====" << RESET << endl;
397         cout << " 1. Nap tu file nhi phan (. dat) – Phuc hoi trang thai
cu\n";
398         cout << " 2. Nap tu file van ban (. txt) – Nhap moi hang loat\n
";
399         cout << " 0. Quay lai Menu chinh\n";
400         cout << YELLOW << "
===== " << RESET
<< endl;
401
402         int subChoice;
403         cout << "Nhap lua chon cua ban: ";
404         cin >> subChoice;
405         cin . ignore ( numeric_limits<streamsize >::max(), '\n');

```



```

406
407         if (subChoice == 0) break;
408
409         string fileName;
410         if (subChoice == 1) {
411             cout << "\n[Lưu y] Nạp file .dat sẽ khôi phục toàn bộ cấu
trúc và ID đã lưu.\n";
412             cout << "Nhập tên file ghi phả (VD: GiaPha_Test.dat): ";
413             getline (cin, fileName);
414             if (!fileName.empty()) {
415                 LoadTreeFromFile(fileName);
416             }
417         }
418         else if (subChoice == 2) {
419             cout << "\n[Lưu y] File .txt phải dùng định dạng: Cha, Con,
Giới Tính, Nam Sinh, Ngày Sinh\n";
420             cout << "Nhập tên file văn bản (VD: NhapLieu.txt): ";
421             getline (cin, fileName);
422             if (!fileName.empty()) {
423                 ImportFromTextFile(fileName);
424             }
425         }
426         else {
427             cout << RED << "Lựa chọn không hợp lệ!" << RESET << endl;
428         }
429
430         cout << "\nẤn [Enter] để tiếp tục ... ";
431         cin.get();
432     } while (askToContinue("thao tác nạp file"));
433     break;
434 }
435
436 case 8: {
437     clearScreen();
438     FamilyStats s;
439     CalculateStats (root, 1, s);
440

```

```

441         cout << YELLOW << BOLD << "===== THONG KE
GIA PHA =====" << RESET << endl;
442         cout << " 1. Quy mo dong ho: " << BOLD << s.totalMembers + s.
totalSpouses << " nguoi" << RESET << endl;
443         cout << "    - Huyet thong: " << s.totalMembers << endl;
444         cout << "    - Phu nhan: " << s.totalSpouses << endl;
445         cout << " 2. So doi (Generations): " << BOLD << s.maxGen <<
RESET << endl;
446         cout << " 3. Co cau gioi tinh: " << DUT_BLUE << "Nam: " << s.
maleCount
447         << RESET << " | " << RED << "Nu: " << s.femaleCount << RESET
<< endl;
448         cout << " 4. Tinh trang: " << "Con song: " << (s.totalMembers -
s.deceasedCount)
449         << " | Da khuat: " << s.deceasedCount << endl;
450
451         cout << "\n" << BOLD << "—— bieu do phan bo thanh vien theo doi
——" << RESET << endl;
452         for (int i = 1; i <= s.maxGen; i++) {
453             cout << " Doi " << (i < 10 ? "0" : "") << i << ": ";
454             for (int j = 0; j < s.genDistribution[i]; j++) cout << "*";
455             cout << " (" << s.genDistribution[i] << ")" << endl;
456         }
457         cout << YELLOW << "
===== " << RESET <<
endl;
458
459         cout << "\nNhan Enter de quay lai menu...";
460         cin.ignore(); cin.get();
461         break;
462     }
463
464     case 0:
465         cout << "Dang don dep bo nho ... Tam biet!\n";
466         FreeTree(root);
467         break;
468

```

```
469         default :  
470             cout << "Lua chon khong hop le !\n";  
471             break;  
472         }  
473     } while (luaChon != 0);  
474  
475     return 0;  
476 }
```

B. Sơ đồ cấu trúc các hàm chính của chương trình

